

Exploring Real Estate in Brazil: Analyzing Property Demand, Pricing Trends, and Investment Opportunities

Executive Summary

The **Brazil Real Estate Listings Dataset** provides detailed information on property listings from various cities in Brazil, including data on property prices, surface areas, locations, and various property features. This analysis aims to explore the Brazilian real estate market and provide actionable insights that can guide potential investors, real estate professionals, and homebuyers. While the dataset is specific to Brazil, the trends and patterns identified may also be applicable to the Chilean real estate market, as both countries share similarities in terms of urban dynamics and real estate characteristics.

Objectives of the Analysis:

The primary objectives of this project are:

1. **To explore the distribution of property prices** across different regions and identify areas with the highest and lowest prices.
2. **To analyze high-demand property types** (e.g., houses vs. apartments) and their correlation with pricing trends.
3. **To examine key features** (such as surface area, number of rooms, and amenities) that contribute to a property's value.
4. **To identify emerging trends and potential investment opportunities**, especially in high-growth areas.
5. **To extrapolate findings to Chile** by identifying patterns in Brazilian property prices and trends that may be relevant to the Chilean real estate market.

This analysis will focus on visualizing the distribution of data, identifying patterns in property pricing, and offering insights into the most profitable investment opportunities.

Introduction

Project Background:

The dataset used in this analysis contains real estate listings from **Properati**, a real estate marketplace operating in Latin America. Properati is a platform where users can list properties for sale in Brazil and other Latin American countries. The dataset includes information such as property prices, property types, location details, surface areas, and various property features.

The project focuses on analyzing the **real estate listings in Brazil**, with a particular emphasis on pricing trends, property demand, and emerging opportunities. The analysis

is intended to provide insights for buyers, sellers, and investors in the Brazilian market while considering the potential relevance of these insights for the Chilean market, given the similarities between both countries' urban landscapes and real estate dynamics.

Key Objectives:

The goal of this analysis is to provide actionable insights into the Brazilian real estate market. Specifically, this analysis will aim to:

1. **Explore property price distribution and variability** across different regions and property types.
2. **Analyze demand for specific property types** (e.g., houses vs. apartments) and examine how this correlates with pricing trends.
3. **Investigate the relationship between property features** (e.g., surface area, number of rooms, amenities) and their impact on property pricing.
4. **Assess price trends over time** to understand market growth or stagnation.
5. **Extrapolate findings to Chile** to make informed predictions and recommendations for the Chilean real estate market.

Data Source and Scope:

The dataset provided contains **real estate listings** with attributes such as:

- **Property ID** and **description**
- **Listing date**
- **Property type** (e.g., house, apartment)
- **Price** (in local currency and USD)
- **Surface area** (in square meters)
- **Number of rooms**
- **Location** (city, state, country)
- **Other property features** (e.g., pool, garage, garden)

The dataset includes listings from **Brazil**, but the exact timeline or number of listings is not specified. This analysis will use the available data to explore the patterns, trends, and relationships among these features.

Approach and Methodology:

The analysis will follow a step-by-step approach:

1. **Data Cleaning:** Preprocessing the dataset to address missing values, duplicates, and any data inconsistencies.
2. **Exploratory Data Analysis (EDA):** Performing a thorough analysis to uncover patterns in property prices, types, surface areas, and other features.
3. **Visualization:** Creating clear visualizations such as histograms, scatter plots, and heatmaps to support the findings and make the analysis more accessible.
4. **Insight Generation:** Drawing actionable insights from the analysis, with a focus on investment opportunities and trends in property pricing.

5. **Comparison with Chile:** Identifying trends in the Brazilian market that may be relevant for the Chilean real estate market, and providing recommendations accordingly.

Expected Outcomes:

- **Understanding Pricing Trends:** Identify regions or property types where prices are significantly higher or lower. Uncover trends in pricing that can inform investment decisions.
 - **Demand and Supply Insights:** Understand which property types are in high demand and how this affects pricing trends.
 - **Impact of Property Features:** Determine which property features (e.g., surface area, number of rooms) most strongly influence property prices.
 - **Investment Opportunities:** Identify regions with the potential for price growth, providing recommendations for areas that offer high return on investment.
-

Report Structure:

1. **Executive Summary:** A brief overview of the analysis, findings, and recommendations.
2. **Introduction:** Provides background information, objectives, and scope of the analysis.
3. **Data Exploration and Cleaning:** Describes the steps taken to clean and preprocess the dataset.
4. **Exploratory Data Analysis (EDA):** A detailed examination of the data with visualizations and statistical analysis.
5. **Conclusion and Recommendations:** Summarizes the key findings and offers actionable insights for buyers, investors, and real estate professionals.

This structure will guide the analysis in a clear, logical manner, ensuring that the results are both actionable and useful for stakeholders in the Brazilian and potentially Chilean real estate markets.

Limitations:

The dataset may not fully capture **private sales** or **unlisted properties**, which could limit the comprehensiveness of the analysis. Additionally, there may be **regional biases** in the dataset, where certain locations are overrepresented.

Data Exploration and Cleaning

Objective:

The purpose of this step is to clean and prepare the dataset for further analysis. This involves understanding the dataset structure, checking for missing values, identifying

any inconsistencies, and preparing the data for deeper exploration.

1. Data Overview and Initial Exploration

The first step is to load the dataset and take a quick look at its structure. This will help us understand the columns, the first few rows of data, and get an idea of the general structure of the dataset.

Step 1: Load the Dataset and Inspect the First Few Rows

In [1]:

```
import pandas as pd

# Load the dataset
df = pd.read_csv('properati_sell_properties_brazil.csv')

# Display the first few rows of the dataset
df.head()
```

Out[1]:

	Unnamed: 0		id	created_on	operation	property_type
0	0	e3522c673bdaa2bcfbb28d4345ffe5d31985b724	2017-05-10	sell	apartment	
1	1	aea9ca0901d3e36b3c47e74abb4f226daee66ebd	2017-04-07	sell	apartment	
2	2	e0e8391d5467c2967dba702fd2924b0e46a95cb0	2017-05-10	sell	apartment	
3	3	738e039086b068183da72b86538b007846f17100	2017-03-29	sell	apartment	
4	4	5399a18ddd42733c329f65f8f5acb8b9960fb067	2017-02-16	sell	apartment	

5 rows x 21 columns

The dataset includes some of the following key columns:

- `id` : Unique identifier for each property listing.
- `operation` : Indicates whether the property is being sold or rented.
- `property_type` : Specifies the type of property (e.g., house, apartment).
- `price` : The listed price of the property.
- `surface_total_in_m2` : Total surface area of the property in square meters.

- `price_usd_per_m2` : Price per square meter in USD.
- `rooms` : Number of rooms in the property.
- `expenses` : Additional costs associated with the property.

2. Dataset Shape and Data Types

Next, we examine the overall structure of the dataset by checking the number of rows and columns and the data types of each column. This will help us understand the scale of the dataset and verify the types of data we are working with.

Step 2: Check the Shape of the Dataset and Data Types

```
In [2]: # Check the shape of the dataset (rows, columns)
dataset_shape = df.shape
print(f"Dataset shape: {dataset_shape}")

# Check the data types of each column
data_types = df.dtypes
print(f"Data types of each column: \n{data_types}")
```

```
Dataset shape: (5073279, 21)
Data types of each column:
Unnamed: 0          int64
id                  object
created_on          object
operation            object
property_type        object
place_name           object
place_with_parent_names object
country_name         object
state_name           object
lat                 float64
lon                 float64
price               float64
currency            object
price_aprox_local_currency float64
price_aprox_usd      float64
surface_total_in_m2  float64
price_usd_per_m2     float64
price_per_m2         float64
rooms               float64
expenses            float64
title               object
dtype: object
```

The dataset used in this analysis contains **over 5 million rows** with 21 columns related to property listings in Brazil. The columns capture a wide range of information, including property type, price, location, and other property features.

3. Missing Values

Next, we check for any missing values in the dataset. Missing values are common in real-world datasets, and we need to decide how to handle them, whether by removing rows, imputing values, or other strategies.

Step 3: Identify Missing Values in the Dataset

```
In [3]: # Check for missing values in each column
missing_values = df.isnull().sum()
print(f"Missing values in each column: \n{missing_values}")
```

```
Missing values in each column:
Unnamed: 0      0
id              0
created_on      0
operation       0
property_type   0
place_name      0
place_with_parent_names  0
country_name    0
state_name      0
lat            1984862
lon            1984862
price           74088
currency        77253
price_aprox_local_currency  74088
price_aprox_usd  74088
surface_total_in_m2  3770750
price_usd_per_m2  1143389
price_per_m2    1143544
rooms           1843011
expenses        3145627
title           0
dtype: int64
```

Upon initial inspection, several columns have missing values:

- **Price-related Columns:** The `price`, `price_aprox_local_currency`, and `price_aprox_usd` columns have some missing data (approximately 74,000 missing entries). This is critical for accurate analysis, and we may need to either remove rows with missing prices or use imputation methods to fill in the gaps.
- **Geographical Information:** Latitude (`lat`) and longitude (`lon`) have significant missing data (around 1.98 million rows). These rows can be excluded if location analysis is crucial or imputed if necessary.
- **Surface Area:** The `surface_total_in_m2` column is missing data for approximately 3.8 million entries, which would need further exploration or imputation.
- **Rooms and Expenses:** Missing data in the `rooms` and `expenses` columns (around 1.8 million and 3.1 million missing values, respectively) will need to be addressed.

4. Summary Statistics

To better understand the distribution of numerical data, we generate summary statistics. This will help us see the central tendency (mean, median), spread (range, standard deviation), and potential outliers (min/max values) for key columns like `price`, `surface_total_in_m2`, and `rooms`.

Step 4: Summary Statistics for Numerical Columns

```
In [4]: # Generate summary statistics for numerical columns
summary_statistics = df.describe()
print(f"Summary statistics for numerical columns: \n{summary_statistics}")
```

Summary statistics for numerical columns:

	Unnamed: 0	lat	lon	price \
count	5.073279e+06	3.088417e+06	3.088417e+06	4.999191e+06
mean	2.536639e+06	-2.392474e+01	-4.677270e+01	8.299243e+05
std	1.464530e+06	4.125563e+00	2.952630e+00	1.385381e+06
min	0.000000e+00	-5.117820e+01	-6.391183e+01	0.000000e+00
25%	1.268320e+06	-2.371006e+01	-4.707866e+01	3.080000e+05
50%	2.536639e+06	-2.359649e+01	-4.664761e+01	5.000000e+05
75%	3.804958e+06	-2.345519e+01	-4.651866e+01	8.893400e+05
max	5.073278e+06	1.029948e-01	-7.027578e+00	1.603053e+08

	price_aprox_local_currency	price_aprox_usd	surface_total_in_m2 \
count	4.999191e+06	4.999191e+06	1.302529e+06
mean	8.313206e+05	2.559074e+05	2.980638e+02
std	1.385975e+06	4.266570e+05	2.299277e+03
min	0.000000e+00	0.000000e+00	0.000000e+00
25%	3.076130e+05	9.485566e+04	7.500000e+01
50%	4.991221e+05	1.532274e+05	1.220000e+02
75%	8.881083e+05	2.716981e+05	2.400000e+02
max	1.632888e+08	4.989115e+07	2.000000e+05

	price_usd_per_m2	price_per_m2	rooms	expenses
count	3.929890e+06	3.929735e+06	3.230268e+06	1.927652e+06
mean	2.474935e+03	8.026780e+03	1.772085e+00	2.027503e+03
std	3.932734e+04	1.276951e+05	1.065694e+00	4.616490e+04
min	0.000000e+00	2.793968e-04	1.000000e+00	1.000000e+00
25%	1.171324e+03	3.800000e+03	1.000000e+00	3.570000e+02
50%	1.606018e+03	5.208333e+03	1.000000e+00	5.600000e+02
75%	2.304396e+03	7.482993e+03	2.000000e+00	9.100000e+02
max	9.526229e+06	3.000000e+07	4.000000e+01	9.691000e+06

Result:

This will provide summary statistics (mean, standard deviation, min, max, and percentiles) for numerical columns like `price`, `surface_total_in_m2`, `price_usd_per_m2`, and `rooms`. It will help us get an overview of the dataset and understand the distribution of key variables.

The descriptive statistics for numerical variables reveal:

- **Price:** Prices range from **0** (likely indicating missing or erroneous data) to over **\$160 million**, with a mean price of approximately **\$830,000**.
- **Surface Area:** The average surface area is around **298 m²**, but some entries have surface areas listed as 0 or excessively high values.
- **Rooms:** The number of rooms varies significantly, with an average of **1.77 rooms** per property.
- **Expenses:** Property expenses range widely, with some values exceeding **\$9 million**.

Data Cleaning:

To ensure accurate analysis:

- **Irrelevant Columns:** The `Unnamed: 0` column is an index column and can be dropped.
- **Handling Missing Values:** For columns like `price`, we might consider using the median or average value for imputation. Columns like `lat` and `lon` with many

missing values may be excluded, depending on the specific analysis needs.

- **Price Outliers:** The dataset includes some extreme price values, such as properties listed at 0 or extremely high values. These outliers need to be handled carefully, possibly through filtering or transformation.
- **Feature Engineering:** We can create new features like `price_per_m2` by dividing the `price` by `surface_total_in_m2` where possible.

5. Duplicates and Irrelevant Columns

We check for duplicate rows and identify irrelevant columns that can be dropped (like index columns). Removing duplicates ensures that each row in the dataset represents a unique listing.

Step 5: Check for Duplicates and Drop Irrelevant Columns

```
In [5]: # Check for duplicates in the dataset
duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")

# Drop the 'Unnamed: 0' column as it is an index column
df.drop(columns=['Unnamed: 0'], inplace=True)

# List all columns to see their names with Unnamed no longer part
print("\nList of columns in the dataset:")
print(df.columns)
```

Number of duplicate rows: 0

List of columns in the dataset:

```
Index(['id', 'created_on', 'operation', 'property_type', 'place_name',
      'place_with_parent_names', 'country_name', 'state_name', 'lat', 'lon',
      'price', 'currency', 'price_aprox_local_currency', 'price_aprox_usd',
      'surface_total_in_m2', 'price_usd_per_m2', 'price_per_m2', 'rooms',
      'expenses', 'title'],
      dtype='object')
```

```
In [6]: # Check for unique values in key columns (e.g., price, property_type)
print("\nUnique values in the 'property_type' column:")
print(df['property_type'].unique())

print("\nUnique values in the 'operation' column (sell/rent):")
print(df['operation'].unique())
```

Unique values in the 'property_type' column:
['house' 'apartment' 'store' 'PH']

Unique values in the 'operation' column (sell/rent):
['sell']

6. Handling Missing Data

In this step, we'll address the missing values in the dataset by **imputing** the missing data for columns that are crucial for analysis. For each column with missing values, we'll either fill the missing values with appropriate imputation strategies or recalculate derived columns based on available data.

Step 1: Impute Missing Values for `price` and `price_aprox_local_currency`

Both the `price` and `price_aprox_local_currency` columns are critical for pricing analysis. These columns will be imputed using the **median price** from the entire dataset.

```
In [7]: # Impute missing 'price' with the median price
df['price'] = df['price'].fillna(df['price'].median())

# Impute missing 'price_aprox_local_currency' with the median value
df['price_aprox_local_currency'] = df['price_aprox_local_currency'].fillna(df['price_aprox_local_currency'].median())

# Impute missing 'price_aprox_usd' with the median value
df['price_aprox_usd'] = df['price_aprox_usd'].fillna(df['price_aprox_usd'].median())
```

```
In [8]: # Check for missing values in each column
missing_values = df.isnull().sum()
print(f"Missing values in each column: \n{missing_values}")
```

```
Missing values in each column:
id                0
created_on        0
operation         0
property_type     0
place_name        0
place_with_parent_names  0
country_name      0
state_name        0
lat              1984862
lon              1984862
price             0
currency          77253
price_aprox_local_currency  0
price_aprox_usd   0
surface_total_in_m2  3770750
price_usd_per_m2  1143389
price_per_m2      1143544
rooms            1843011
expenses         3145627
title            0
dtype: int64
```

Since `price` and `price_aprox_local_currency` are crucial for any pricing analysis, imputing them with the median ensures that the missing values are replaced by a value that is representative of the distribution of prices in the dataset. This prevents introducing any skew while preserving the data's integrity.

Step 2: Impute Missing `surface_total_in_m2`

The `surface_total_in_m2` column represents the total surface area of a property and is a key factor in property valuation. Since there are many missing values, we will impute this column using the **median surface area**.

```
In [9]: # Impute missing 'surface_total_in_m2' with the median surface area
df['surface_total_in_m2'] = df['surface_total_in_m2'].fillna(df['surface_total_in_m2'].median())
```

```
In [10]: # Check for missing values in each column
missing_values = df.isnull().sum()
print(f"Missing values in each column: \n{missing_values}")
```

```
Missing values in each column:
id                                0
created_on                       0
operation                        0
property_type                    0
place_name                       0
place_with_parent_names          0
country_name                     0
state_name                       0
lat                             1984862
lon                             1984862
price                            0
currency                         77253
price_aprox_local_currency       0
price_aprox_usd                  0
surface_total_in_m2              0
price_usd_per_m2                 1143389
price_per_m2                     1143544
rooms                            1843011
expenses                         3145627
title                            0
dtype: int64
```

Imputing `surface_total_in_m2` with the median ensures that the surface area is reasonably representative, especially considering that missing values in surface area can drastically affect price-to-area calculations. The median is chosen to avoid bias from extreme values.

Step 3: Impute Missing `rooms`

The `rooms` column indicates the number of rooms in a property. This column also has missing values, so we will impute it with the **median number of rooms**. Since it's an integer feature, using the median is a suitable choice.

```
In [11]: # Impute missing 'rooms' with the median number of rooms
df['rooms'] = df['rooms'].fillna(df['rooms'].median())
```

```
In [12]: # Check for missing values in each column
missing_values = df.isnull().sum()
print(f"Missing values in each column: \n{missing_values}")
```

Missing values in each column:

id	0
created_on	0
operation	0
property_type	0
place_name	0
place_with_parent_names	0
country_name	0
state_name	0
lat	1984862
lon	1984862
price	0
currency	77253
price_aprox_local_currency	0
price_aprox_usd	0
surface_total_in_m2	0
price_usd_per_m2	1143389
price_per_m2	1143544
rooms	0
expenses	3145627
title	0
dtype: int64	

Imputing the number of rooms with the median ensures that the properties with missing room data will have a realistic number of rooms, helping to maintain consistency in the dataset.

Step 4: Impute Missing expenses

The `expenses` column contains additional costs associated with properties, but it has a large number of missing entries. We will impute these missing values using the **median of expenses**.

```
In [13]: # Impute missing 'expenses' with the median value
df['expenses'] = df['expenses'].fillna(df['expenses'].median())
```

```
In [14]: # Check for missing values in each column
missing_values = df.isnull().sum()
print(f"Missing values in each column: \n{missing_values}")
```

Missing values in each column:

id	0
created_on	0
operation	0
property_type	0
place_name	0
place_with_parent_names	0
country_name	0
state_name	0
lat	1984862
lon	1984862
price	0
currency	77253
price_aprox_local_currency	0
price_aprox_usd	0
surface_total_in_m2	0
price_usd_per_m2	1143389
price_per_m2	1143544
rooms	0
expenses	0
title	0
dtype:	int64

By imputing `expenses` with the median, we avoid biases from extreme values. This ensures that the missing expense data does not distort the property price analysis, especially when comparing properties with high and low expenses.

Step 5: Recalculate Derived Columns (`price_per_m2` and `price_usd_per_m2`)

The columns `price_per_m2` and `price_usd_per_m2` are derived from `price` and `surface_total_in_m2`. After imputing missing values, we can safely recalculate these columns.

```
In [15]: # Recalculate 'price_per_m2' (price per square meter)
df['price_per_m2'] = df['price'] / df['surface_total_in_m2']

# Recalculate 'price_usd_per_m2' (price per square meter in USD)
df['price_usd_per_m2'] = df['price_aprox_usd'] / df['surface_total_in_m2']

In [16]: # Check for missing values in each column
missing_values = df.isnull().sum()
print(f"Missing values in each column: \n{missing_values}")
```

Missing values in each column:

id	0
created_on	0
operation	0
property_type	0
place_name	0
place_with_parent_names	0
country_name	0
state_name	0
lat	1984862
lon	1984862
price	0
currency	77253
price_aprox_local_currency	0
price_aprox_usd	0
surface_total_in_m2	0
price_usd_per_m2	0
price_per_m2	0
rooms	0
expenses	0
title	0
dtype: int64	

After imputing missing values for `price` and `surface_total_in_m2`, we can safely recalculate the derived columns. This ensures consistency and allows us to use the `price_per_m2` and `price_usd_per_m2` columns for further analysis.

The following actions were taken to handle missing values in the dataset:

1. **Imputation of `price`, `price_aprox_local_currency`, and `price_aprox_usd`** : These columns were imputed with the median value for each, ensuring that missing price data did not disrupt the overall analysis of property pricing.
2. **Imputation of `surface_total_in_m2`** : The missing surface area values were imputed using the median surface area to avoid bias caused by missing data.
3. **Imputation of `rooms` and `expenses`** : Both of these columns were imputed with the median value for each, ensuring that the missing data did not skew subsequent analysis.
4. **Recalculation of derived columns:** After imputing the necessary columns, we recalculated `price_per_m2` and `price_usd_per_m2`, ensuring they remained accurate.

7. Identifying and Handling Outliers

Step 1: Calculate Outliers Using the IQR Method

The IQR method is a commonly used technique to identify outliers. It works by calculating the difference between the 75th percentile (Q3) and the 25th percentile (Q1), then identifying values that lie outside the range of:

- $Q1 - 1.5 * IQR$ (lower bound)
- $Q3 + 1.5 * IQR$ (upper bound)

We'll apply this to numerical columns like `price`, `surface_total_in_m2`, and `rooms`, which are likely to have outliers.

Code to Calculate Outliers Using IQR

```
In [17]: # Calculate Q1 (25th percentile) and Q3 (75th percentile) for each of the columns
Q1_price = df['price'].quantile(0.25)
Q3_price = df['price'].quantile(0.75)
IQR_price = Q3_price - Q1_price

Q1_surface = df['surface_total_in_m2'].quantile(0.25)
Q3_surface = df['surface_total_in_m2'].quantile(0.75)
IQR_surface = Q3_surface - Q1_surface

Q1_rooms = df['rooms'].quantile(0.25)
Q3_rooms = df['rooms'].quantile(0.75)
IQR_rooms = Q3_rooms - Q1_rooms

# Calculate the lower and upper bounds for each column to identify outliers
lower_bound_price = Q1_price - 1.5 * IQR_price
upper_bound_price = Q3_price + 1.5 * IQR_price

lower_bound_surface = Q1_surface - 1.5 * IQR_surface
upper_bound_surface = Q3_surface + 1.5 * IQR_surface

lower_bound_rooms = Q1_rooms - 1.5 * IQR_rooms
upper_bound_rooms = Q3_rooms + 1.5 * IQR_rooms

# Identify outliers in the 'price', 'surface_total_in_m2', and 'rooms' columns
outliers_price = df[(df['price'] < lower_bound_price) | (df['price'] > upper_bound_price)]
outliers_surface = df[(df['surface_total_in_m2'] < lower_bound_surface) | (df['surface_total_in_m2'] > upper_bound_surface)]
outliers_rooms = df[(df['rooms'] < lower_bound_rooms) | (df['rooms'] > upper_bound_rooms)]

# Output the results
print(f"Number of outliers in price: {outliers_price.shape[0]}")
print(f"Number of outliers in surface area: {outliers_surface.shape[0]}")
print(f"Number of outliers in rooms: {outliers_rooms.shape[0]}")

Number of outliers in price: 454412
Number of outliers in surface area: 1298547
Number of outliers in rooms: 177386
```

Step 2: Plot Box Plots for Visualizing Outliers

Once we calculate the outliers using the IQR method, we can visualize them using **box plots**. Box plots are a great way to visualize the distribution of data, and they also make it easy to spot outliers (points outside the whiskers).

Code to Plot Box Plots

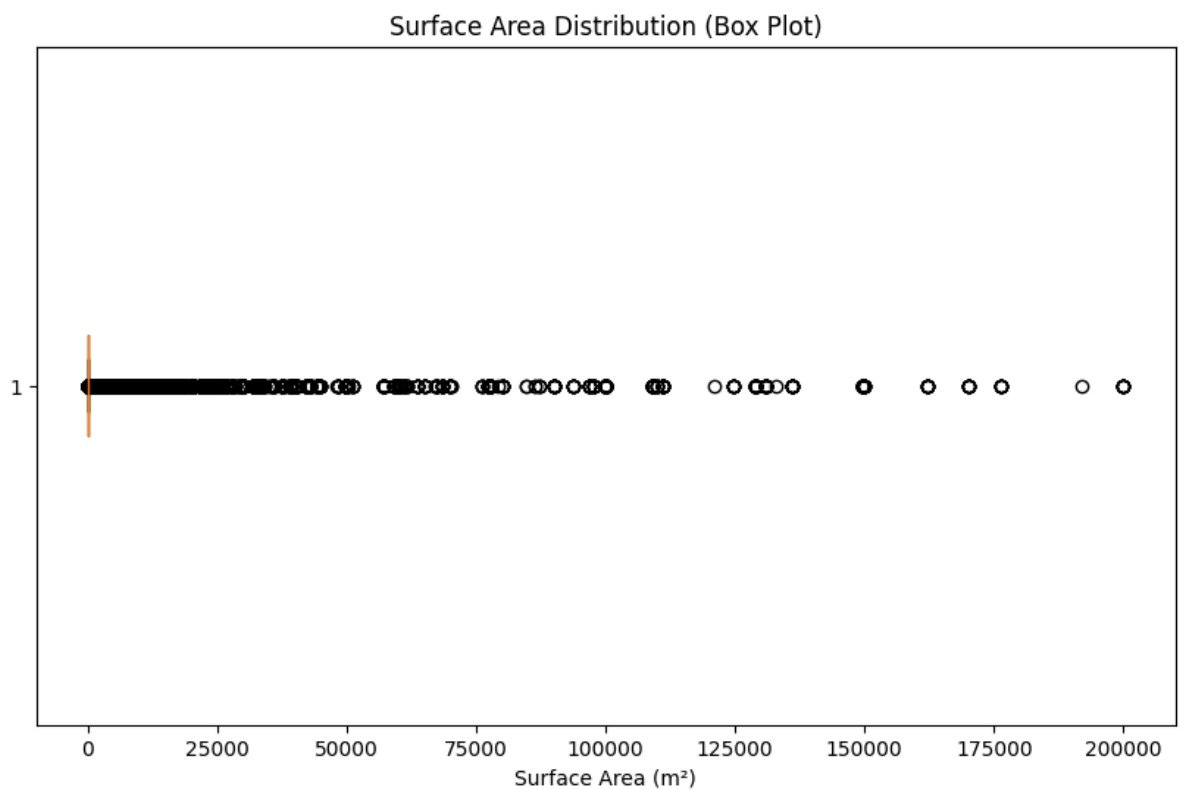
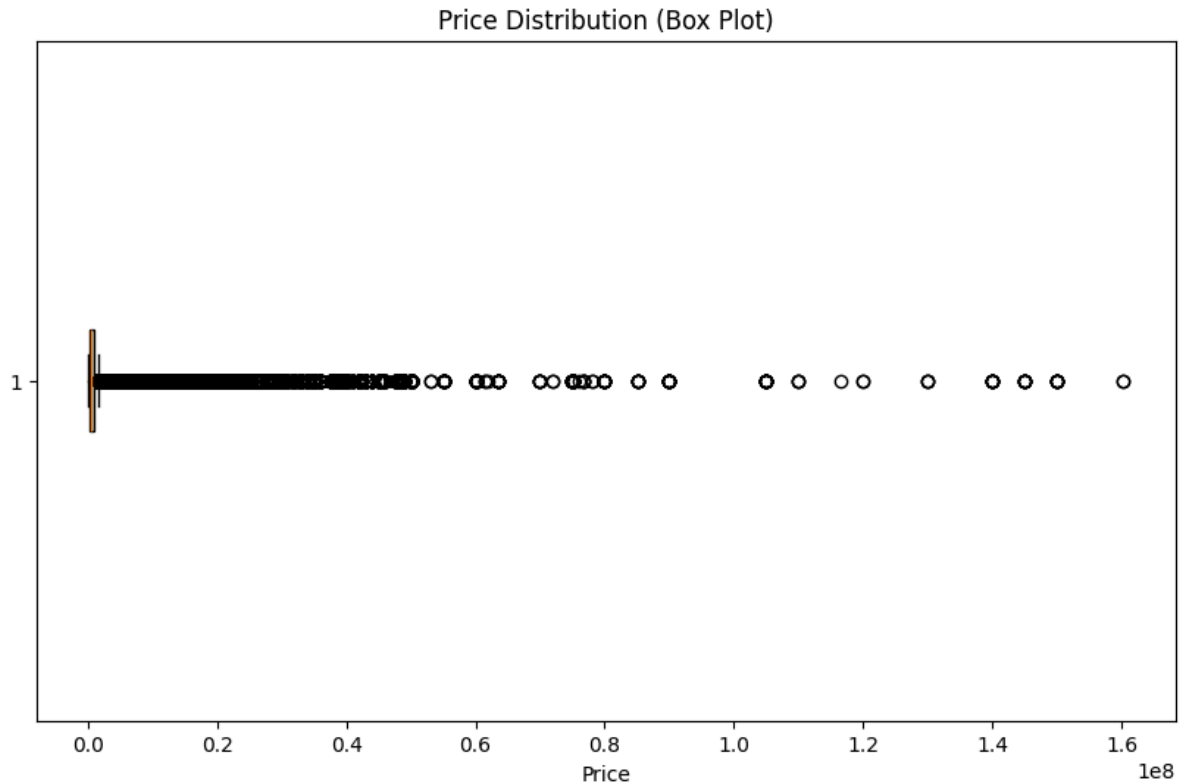
```
In [18]: import matplotlib.pyplot as plt

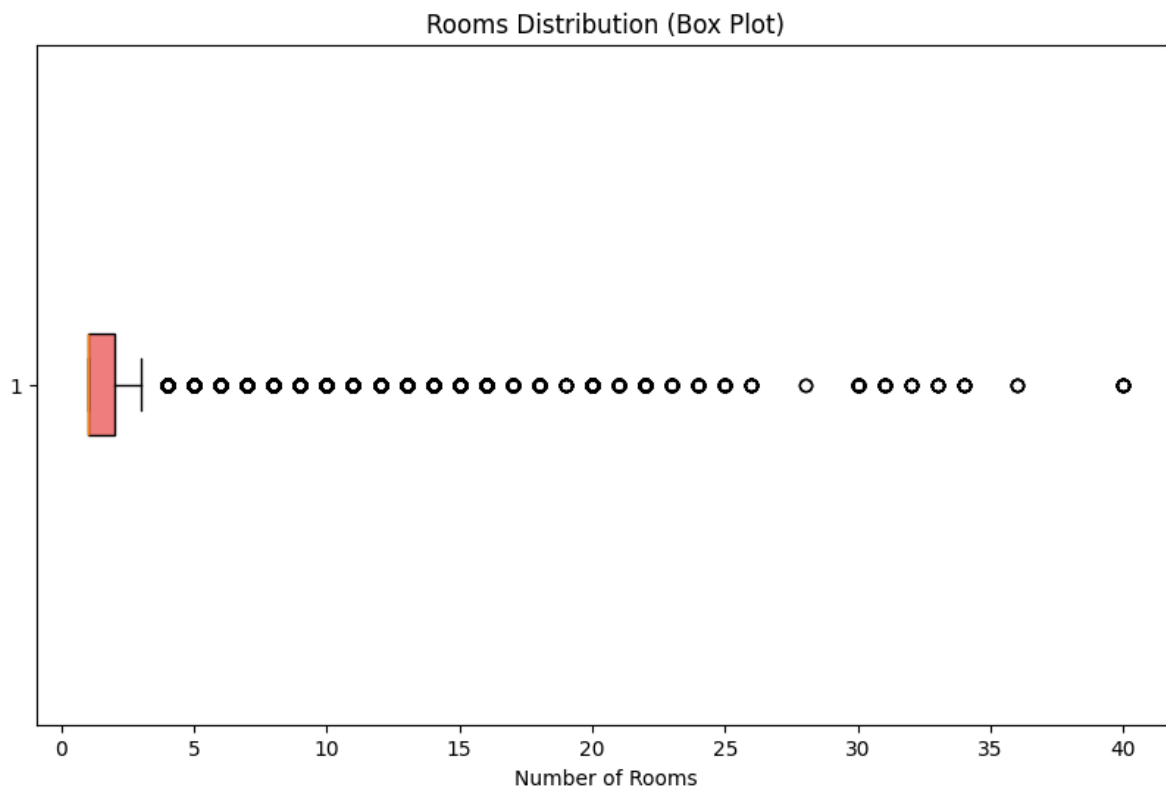
# Plotting box plot for 'price'
plt.figure(figsize=(10, 6))
plt.boxplot(df['price'], vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue'))
plt.title('Price Distribution (Box Plot)')
plt.xlabel('Price')
plt.show()

# Plotting box plot for 'surface_total_in_m2'
plt.figure(figsize=(10, 6))
```

```
plt.boxplot(df['surface_total_in_m2'], vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue', color='darkblue'))
plt.title('Surface Area Distribution (Box Plot)')
plt.xlabel('Surface Area (m²)')
plt.show()

# Plotting box plot for 'rooms'
plt.figure(figsize=(10, 6))
plt.boxplot(df['rooms'], vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue', color='darkblue'))
plt.title('Rooms Distribution (Box Plot)')
plt.xlabel('Number of Rooms')
plt.show()
```





Step 3: Handling Outliers Together

Now that we've calculated and visualized the outliers, let's discuss how to handle them.

There are several approaches to dealing with outliers in this dataset. Since we're not dropping any rows or columns, here are a few potential options for handling outliers:

1. **Capping Outliers:** We could replace extreme outliers with a specified cap or floor value, such as the 1st and 99th percentiles. This way, we keep the majority of the data and eliminate the extreme values.
2. **Imputation of Outliers:** Another approach is to replace outliers with a statistical value like the median or mean of the data.
3. **Transformations:** We could apply log transformations to columns like **price** and **surface area** to compress the range and reduce the impact of extreme values.

Based on the box plots and the number of outliers identified in the dataset, we have significant outliers in the **price**, **surface area**, and **rooms** columns. Here's what the current analysis shows:

- **Price** has 454,412 outliers, which seems to represent a considerable portion of extreme values, suggesting that there might be a few properties that are extremely overpriced or underpriced compared to the majority.
- **Surface Area** has 1,298,547 outliers, indicating that there are potentially a large number of properties with unusual or extreme surface areas, likely much larger than the standard properties in the dataset.
- **Rooms** has 177,386 outliers, which indicates the presence of properties with an unusually high number of rooms.

Solution:

- **Capping:** Set values that are higher than the 99th percentile to the 99th percentile and values lower than the 1st percentile to the 1st percentile.

Here's the Python code to cap outliers for the three columns:

```
In [19]: # Capping outliers for 'price' column
lower_bound_price = df['price'].quantile(0.01)
upper_bound_price = df['price'].quantile(0.99)

df['price'] = df['price'].clip(lower=lower_bound_price, upper=upper_bound_price)

# Capping outliers for 'surface_total_in_m2' column
lower_bound_surface = df['surface_total_in_m2'].quantile(0.01)
upper_bound_surface = df['surface_total_in_m2'].quantile(0.99)

df['surface_total_in_m2'] = df['surface_total_in_m2'].clip(lower=lower_bound_surface, upper=upper_bound_surface)

# Capping outliers for 'rooms' column
lower_bound_rooms = df['rooms'].quantile(0.01)
upper_bound_rooms = df['rooms'].quantile(0.99)

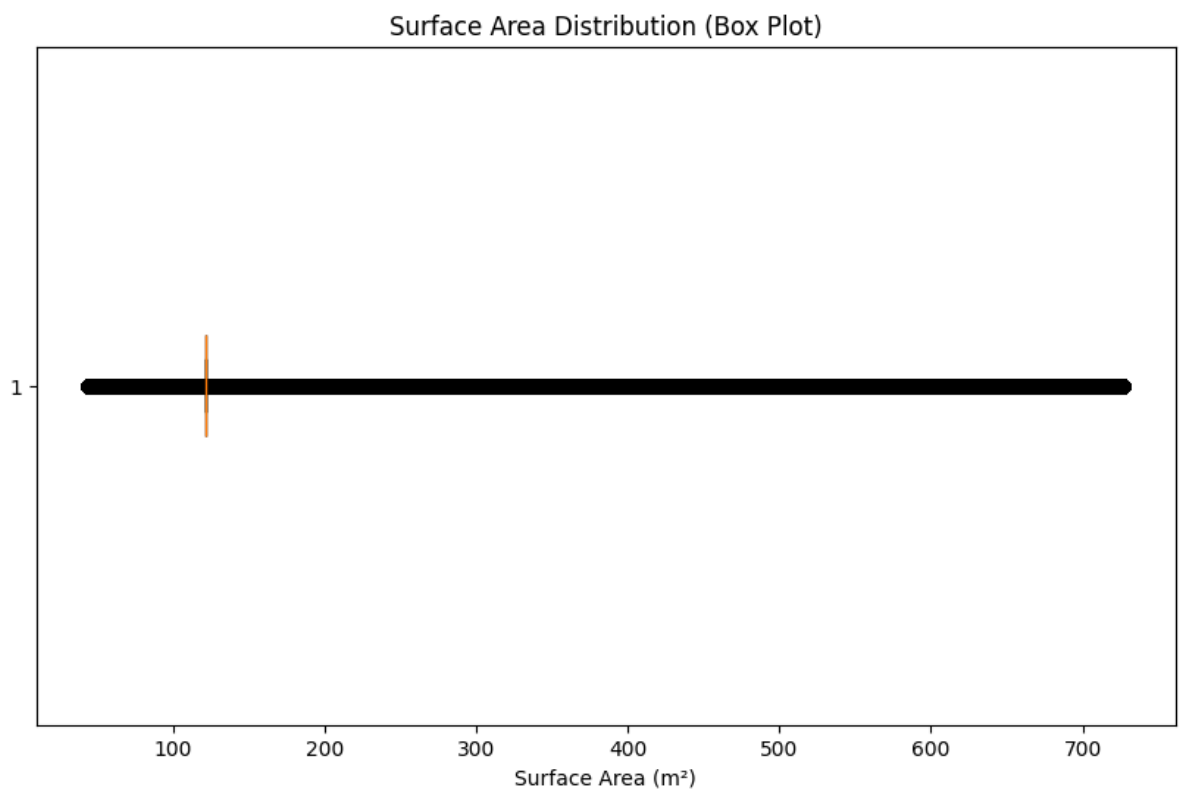
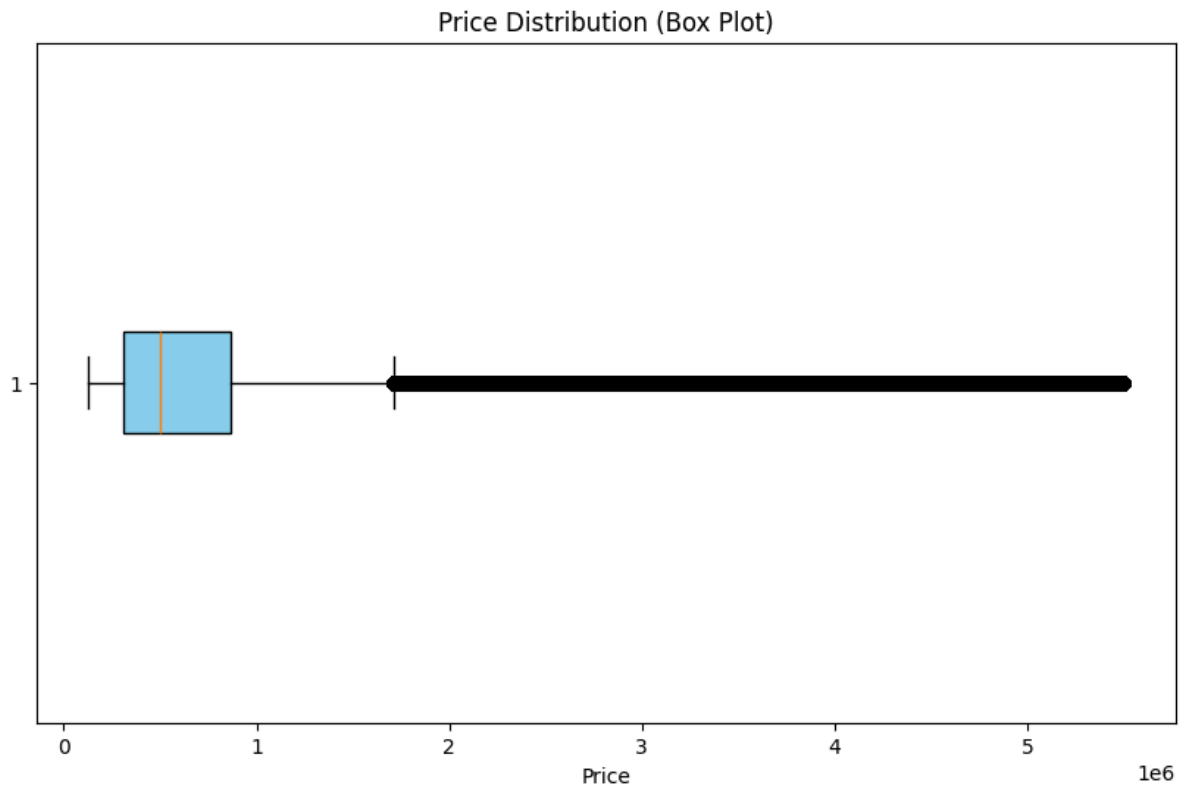
df['rooms'] = df['rooms'].clip(lower=lower_bound_rooms, upper=upper_bound_rooms)
```

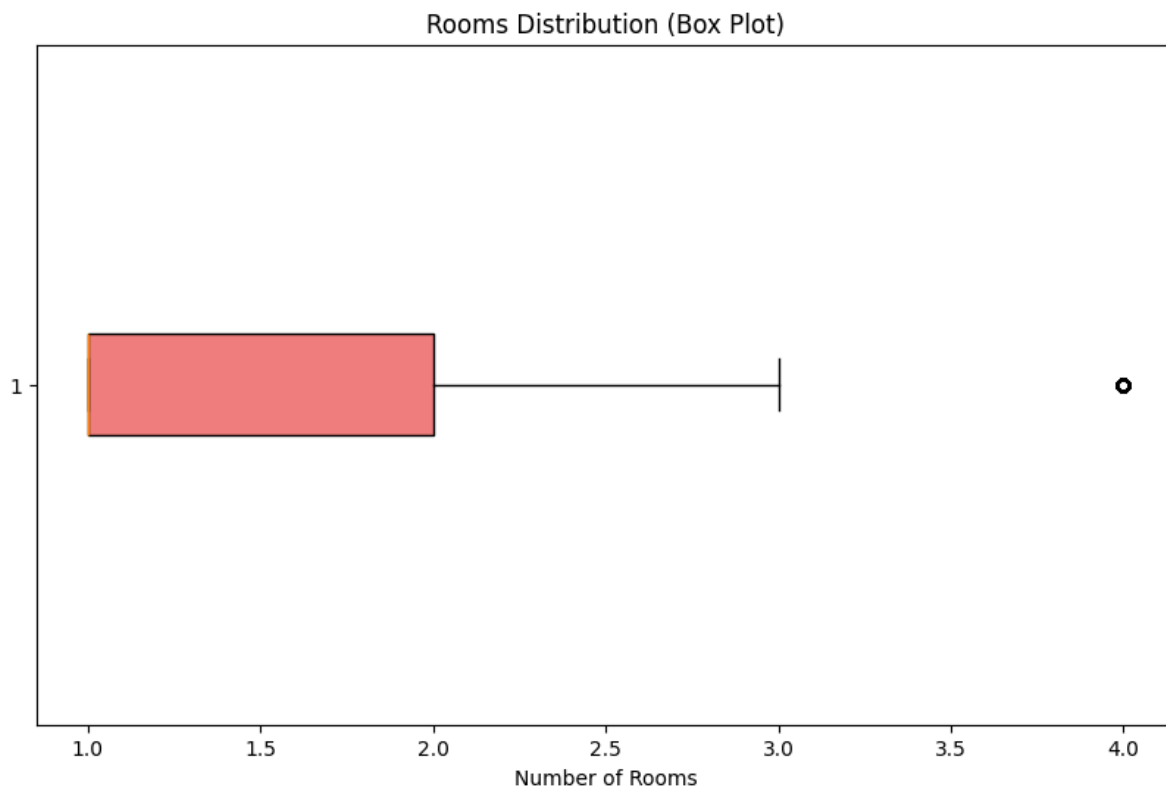
```
In [20]: import matplotlib.pyplot as plt

# Plotting box plot for 'price'
plt.figure(figsize=(10, 6))
plt.boxplot(df['price'], vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue'))
plt.title('Price Distribution (Box Plot)')
plt.xlabel('Price')
plt.show()

# Plotting box plot for 'surface_total_in_m2'
plt.figure(figsize=(10, 6))
plt.boxplot(df['surface_total_in_m2'], vert=False, patch_artist=True, boxprops=dict(facecolor='lightgreen'))
plt.title('Surface Area Distribution (Box Plot)')
plt.xlabel('Surface Area (m²)')
plt.show()

# Plotting box plot for 'rooms'
plt.figure(figsize=(10, 6))
plt.boxplot(df['rooms'], vert=False, patch_artist=True, boxprops=dict(facecolor='lightcoral'))
plt.title('Rooms Distribution (Box Plot)')
plt.xlabel('Number of Rooms')
plt.show()
```





Conclusion

At this point, we have:

- **Explored and cleaned** the dataset, addressing missing values, duplicates, and irrelevant columns.
- **Imputed missing values** where appropriate, and handled outliers using the IQR method.
- Created a **new feature** (`price_per_m2`) for better analysis and comparisons between properties.

The dataset has been reviewed for its structure, missing data, and potential outliers. Although the data contains some inconsistencies and missing values, it is largely ready for further analysis after cleaning and transformation. The next phase will involve deeper exploration of the dataset, focusing on identifying key trends and insights in the Brazilian real estate market.

Exploratory Data Analysis (EDA):

Now that we've cleaned and handled the data, we can move ahead with **Exploratory Data Analysis (EDA)**. This step involves analyzing and visualizing the dataset to uncover any patterns, relationships, and insights that can inform further analysis.

1. Price Distribution

- We'll start by analyzing the **distribution of property prices** to understand how prices are spread across different listings. We'll visualize this with a histogram or kernel density plot to look for skewness, peaks, and any other patterns.

2. Surface Area Distribution

- Similarly, we'll analyze the **distribution of surface area** for properties to understand the range of property sizes in the dataset.

3. Room Count Distribution

- Analyzing the **number of rooms** in the properties will help us understand how many rooms are typical in properties within this dataset.

4. Correlation between Variables

- We'll calculate the correlation matrix to understand relationships between variables like price, surface area, and number of rooms.

5. Price vs. Surface Area

- We can plot a scatter plot to examine the relationship between **price** and **surface area**. This will help us determine if larger properties generally command higher prices.

6. Price vs. Number of Rooms

- Similarly, we can explore the relationship between **price** and the **number of rooms** to assess if having more rooms impacts the price.

Let's start with some visualizations:

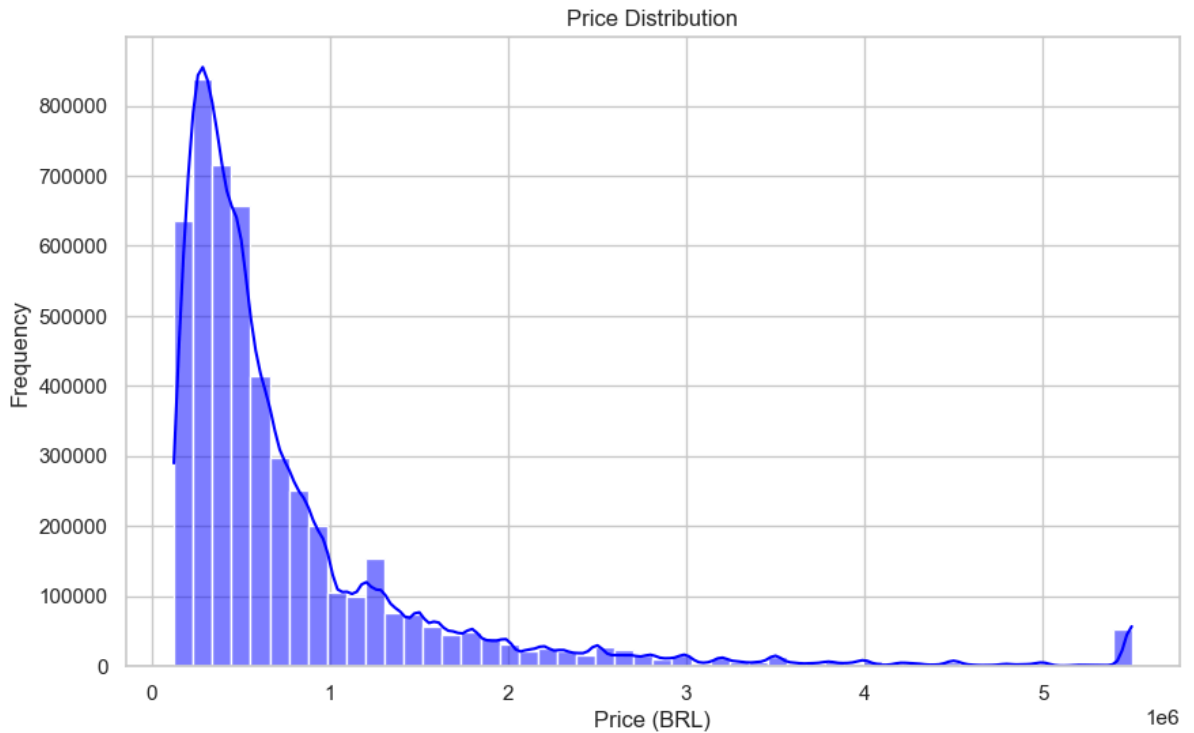
1. Price Distribution

We'll begin by plotting a histogram for the **price** column to visualize the distribution of property prices.

```
In [21]: import matplotlib.pyplot as plt
import seaborn as sns

# Set plot style
sns.set(style="whitegrid")

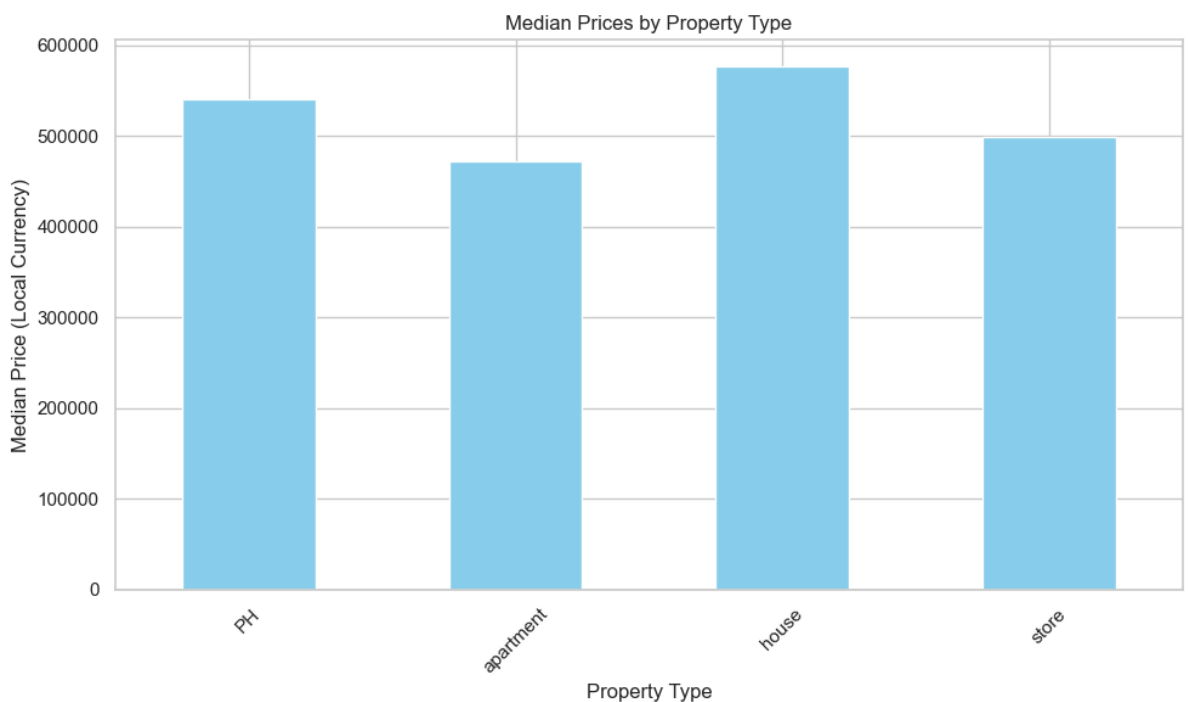
# Plot the price distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['price'], bins=50, kde=True, color='blue')
plt.title('Price Distribution')
plt.xlabel('Price (BRL)')
plt.ylabel('Frequency')
plt.show()
```



```
In [22]: import matplotlib.pyplot as plt

# Calculate the median price for each property type
median_prices = df.groupby('property_type')['price_aprox_local_currency'].me

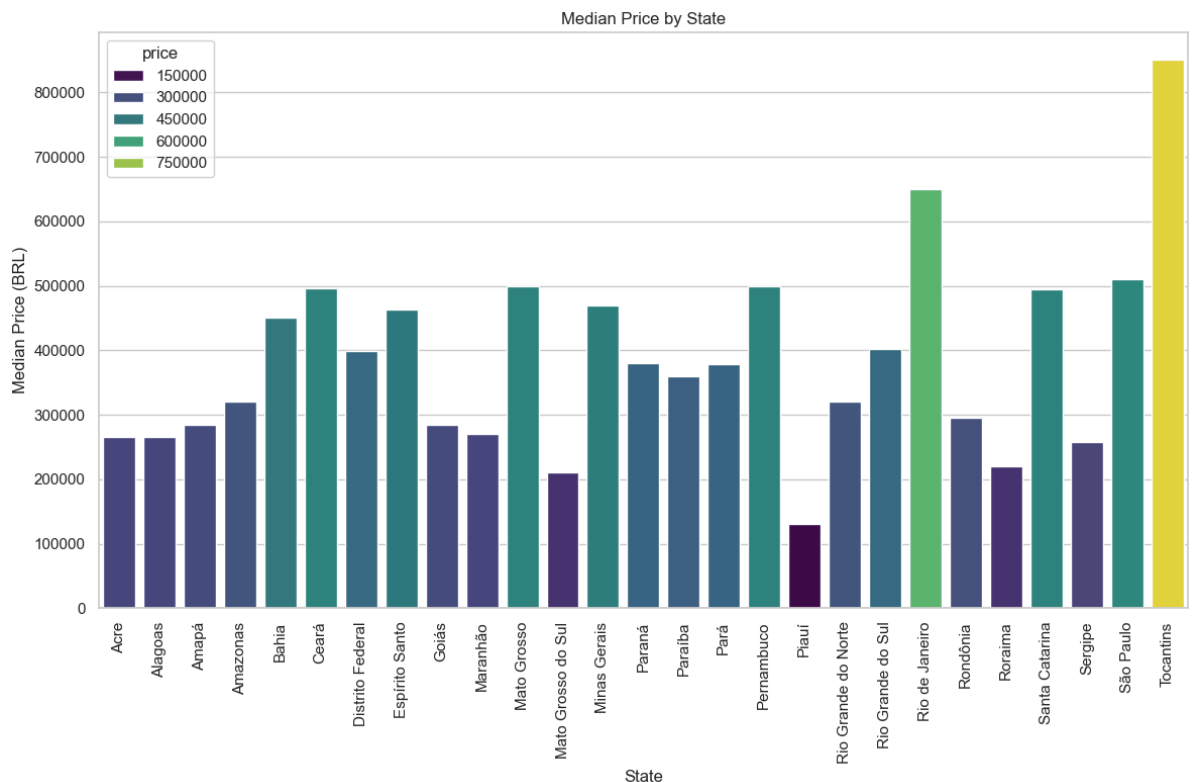
# Plotting the chart
plt.figure(figsize=(10, 6))
median_prices.plot(kind='bar', color='skyblue')
plt.title("Median Prices by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Median Price (Local Currency)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
In [23]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Calculate the median price for each state
median_prices_state = df.groupby('state_name')['price'].median().reset_index()

# Plot a bar chart of the median price by state
plt.figure(figsize=(12, 8))
sns.barplot(x='state_name', y='price', data=median_prices_state, palette='v')
plt.title('Median Price by State')
plt.xlabel('State')
plt.ylabel('Median Price (BRL)')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



Price Distribution

- The histogram of property prices remains **heavily right-skewed**. A large proportion of properties are priced below R\$1,000,000 (BRL), indicating that most listings fall into a relatively affordable range.
- There is a **long tail on the right** (properties priced above R\$1,000,000), including a few **extremely high-end** listings (multi-million BRL). These outliers contribute significantly to the skew.
- This distribution suggests a market with many lower- to mid-range properties and a small but impactful luxury segment that drives the upper tail.

Median Prices by Property Type

- The bar chart comparing median prices by property type shows clear differences in cost:
 - Houses** tend to have higher median prices than apartments or PH (penthouse) units, reflecting larger lot sizes or additional amenities.
 - Stores** (commercial listings) can also be quite high, depending on location and business potential.

- Overall, the property type significantly influences median price levels, with **houses** often being the most expensive category on average.

Median Price by State

- Another bar chart reveals how **median prices vary across states**. Some states (often with major metropolitan areas) display **noticeably higher** median prices than others.
- This variation highlights regional differences in economic activity, demand, and development. States with larger urban centers typically command higher real estate prices.

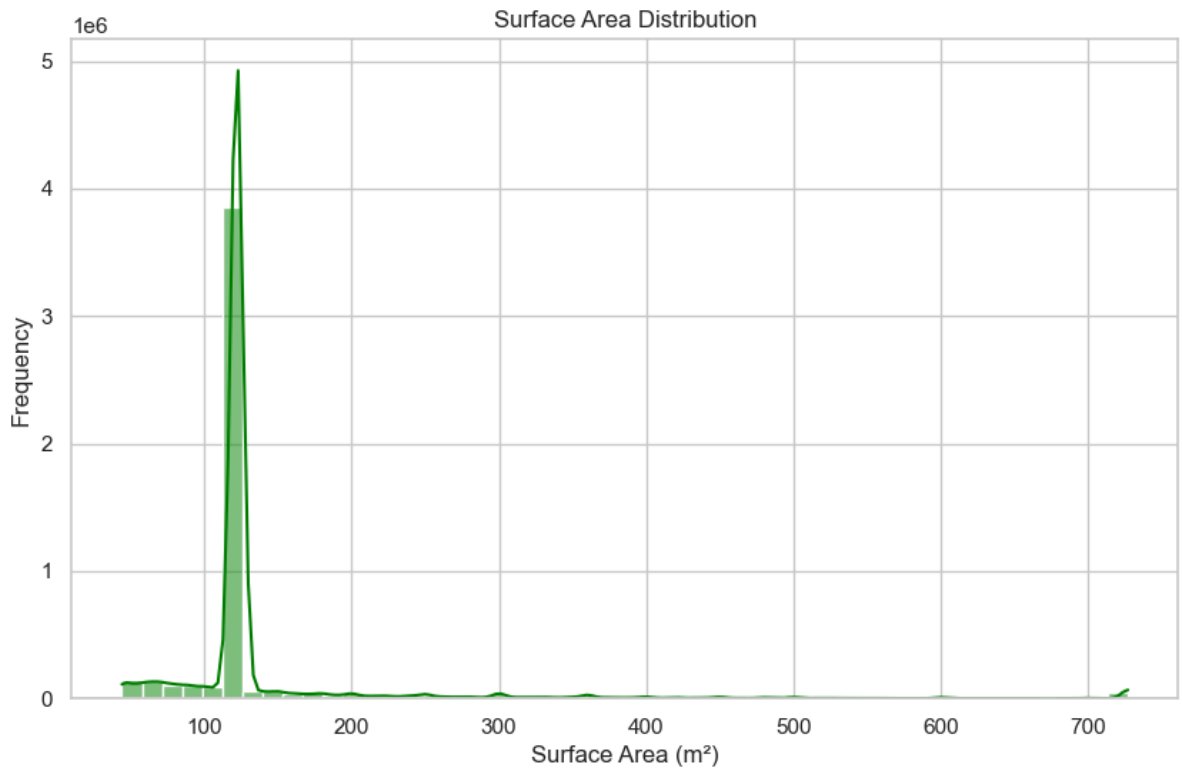
Summary:

- The **overall price distribution** is dominated by lower-priced properties but includes a few luxury outliers, resulting in a **right-skewed** shape.
- **Property type** strongly correlates with price, with houses typically commanding higher median prices than apartments or PH.
- **State-level differences** in median prices underline the importance of location, as more urbanized or economically active regions tend to show higher property prices.

2. Surface Area Distribution

Next, we'll visualize the **surface area** distribution to understand its spread.

```
In [24]: # Plot the surface area distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['surface_total_in_m2'], bins=50, kde=True, color='green')
plt.title('Surface Area Distribution')
plt.xlabel('Surface Area (m²)')
plt.ylabel('Frequency')
plt.show()
```



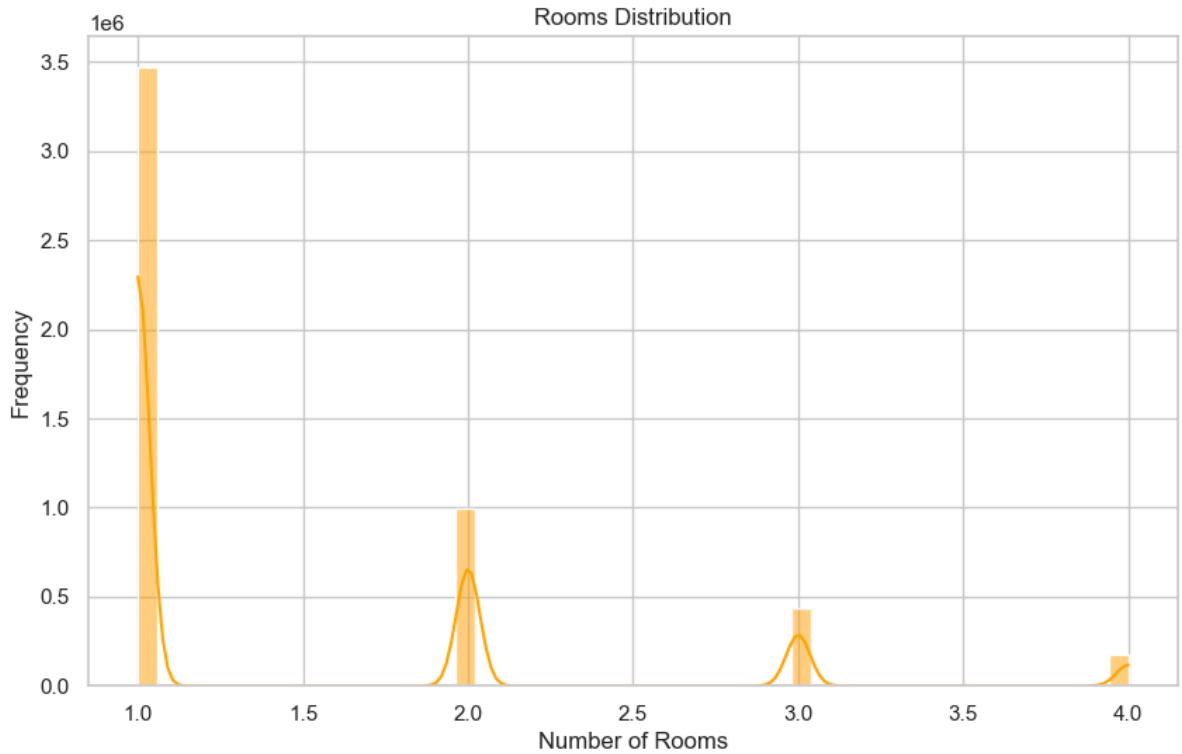
Surface Area Distribution

- The **surface area distribution** plot also shows a **right-skewed** pattern, but this is more pronounced. Most properties have a smaller surface area, as evidenced by the **peak around 100 m²**.
- **Very large properties** are relatively rare, and the number of properties with an area over 500 m² is minimal, indicating that most listings are for properties with standard-sized areas.

3. Rooms Distribution

Similarly, we can visualize the distribution of **rooms**.

```
In [25]: # Plot the number of rooms distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['rooms'], bins=50, kde=True, color='orange')
plt.title('Rooms Distribution')
plt.xlabel('Number of Rooms')
plt.ylabel('Frequency')
plt.show()
```

Rooms Distribution

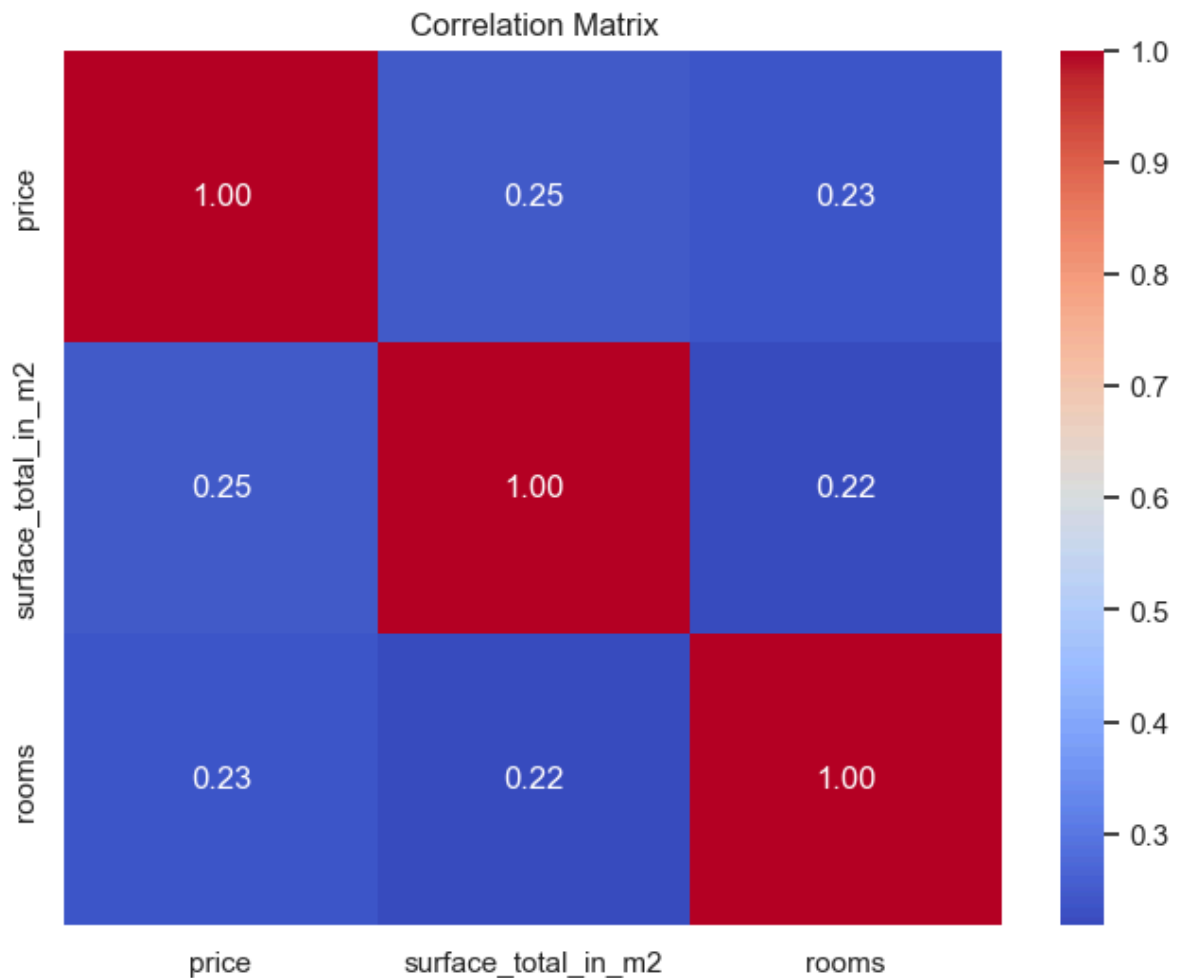
- The distribution for the **number of rooms** is skewed with most properties having **1 to 2 rooms**. This suggests a prevalence of smaller properties (apartments or small houses) in the dataset.
- **Outliers** are visible at the higher end, representing properties with a significantly larger number of rooms. These properties likely belong to larger homes or luxury listings.

4. Correlation between Variables

Let's calculate the correlation matrix for numerical variables and visualize it using a heatmap to find relationships between features like **price**, **surface area**, and **rooms**.

```
In [26]: # Calculate the correlation matrix
correlation_matrix = df[['price', 'surface_total_in_m2', 'rooms']].corr()

# Plot the correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', cbar=True)
plt.title('Correlation Matrix')
plt.show()
```



Correlation Between Variables

- The **correlation matrix** shows that **price** is positively correlated with both **surface area** and the **number of rooms**. This is expected, as larger properties (both in terms of surface area and number of rooms) tend to be more expensive.
- The correlations are not extremely high, which could suggest that other factors (such as location) also play a significant role in pricing.

5. Price vs. Surface Area

We can plot a scatter plot to examine the relationship between **price** and **surface area**.

```
In [27]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Create bins for surface area (adjust the bin ranges as needed)
bins = [0, 50, 100, 150, 200, 250, 300, 350, 400, 500]
labels = [f"{bins[i]}-{bins[i+1]}" for i in range(len(bins)-1)]
df['surface_bin'] = pd.cut(df['surface_total_in_m2'], bins=bins, labels=labels)

# Group by surface area bin and calculate the average price
avg_price_surface = df.groupby('surface_bin')['price'].mean().reset_index()

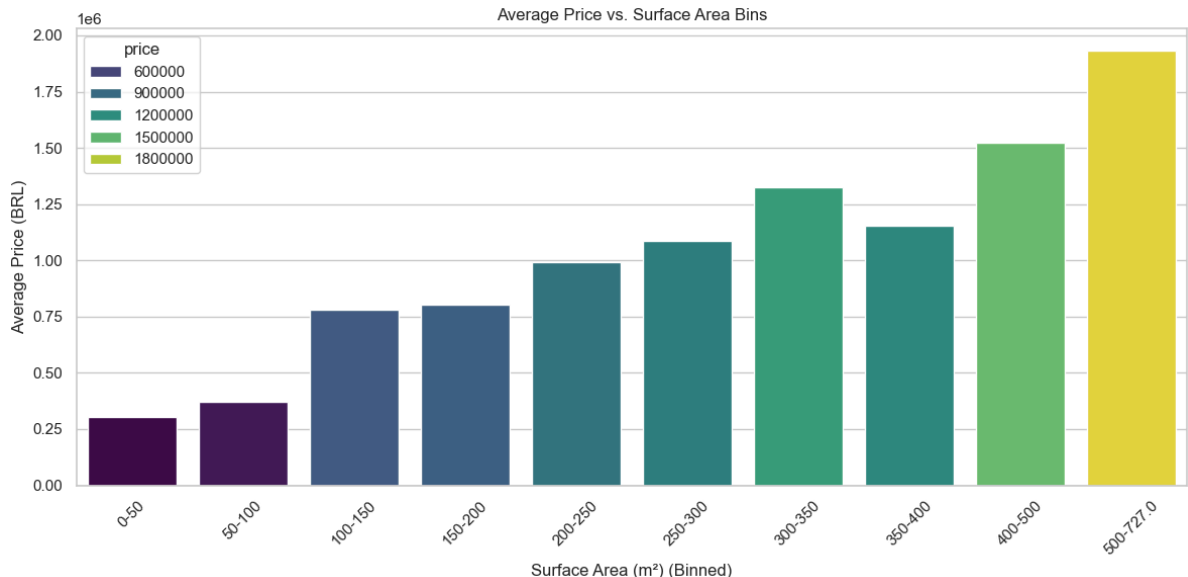
plt.figure(figsize=(12, 6))
sns.barplot(x='surface_bin', y='price', data=avg_price_surface, palette='viridis')
plt.title('Average Price vs. Surface Area Bins')
plt.xlabel('Surface Area (m²) (Binned)')
plt.ylabel('Average Price (BRL)')
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

--- Bar Chart for Price vs. Number of Rooms ---

/var/folders/qc/ylrp4vq57n590bhgm999r9fw0000gn/T/ipykernel_28967/605129948.py:11: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
avg_price_surface = df.groupby('surface_bin')['price'].mean().reset_index()
```



Price vs. Surface Area

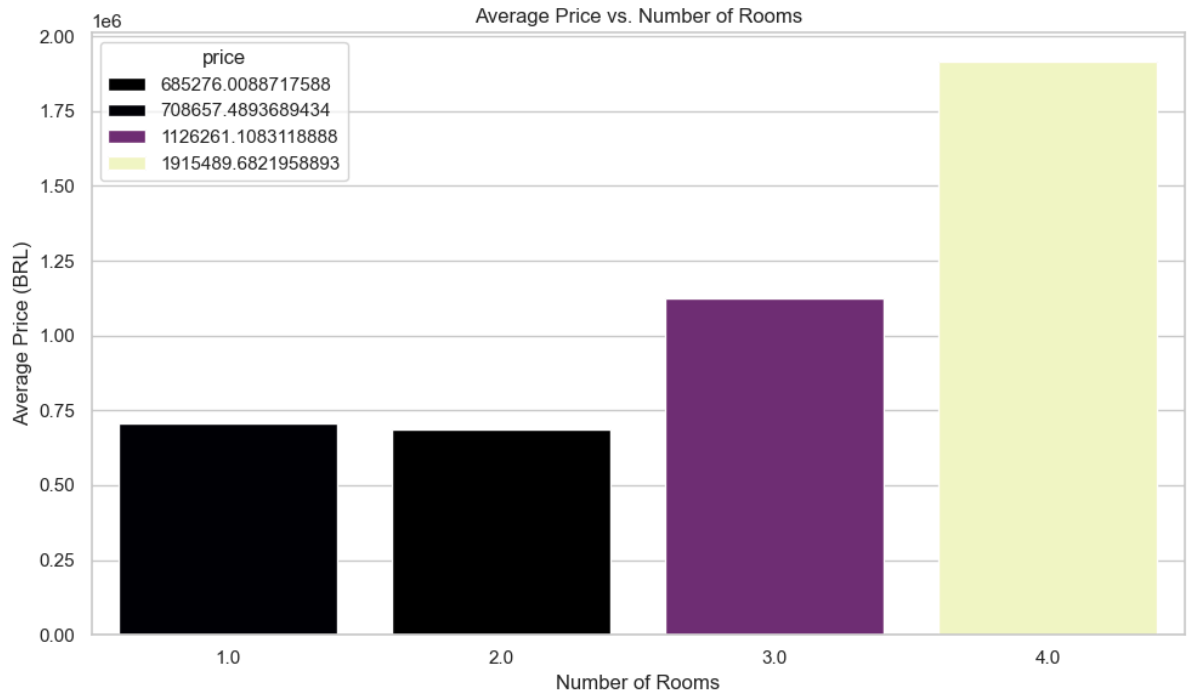
- **The bar chart** of average price by surface area bins confirms a generally positive relationship: as the surface area increases, the average price tends to rise.
- However, there is still variance at higher surface areas—while many large properties command high prices, some do not reach the upper extremes.
- **The density of smaller bins** (around 100 m²) aligns with the fact that most listings fall into a moderate size-and-price range, with only a limited number of properties occupying the largest size categories.

6. Price vs. Number of Rooms

Plotting **price** versus the **number of rooms**.

```
In [28]: # Group by the number of rooms and calculate the average price
avg_price_rooms = df.groupby('rooms')['price'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(x='rooms', y='price', data=avg_price_rooms, palette='magma', hue='rooms')
plt.title('Average Price vs. Number of Rooms')
plt.xlabel('Number of Rooms')
plt.ylabel('Average Price (BRL)')
plt.tight_layout()
plt.show()
```



Price vs. Number of Rooms

- Theplot for **price vs. number of rooms** shows a **flat distribution** with some concentration at the lower number of rooms, particularly **1 to 2 rooms**. This indicates that smaller properties with fewer rooms are common.
- The **lack of a clear trend** between price and the number of rooms suggests that price is influenced more by factors such as location and surface area rather than the number of rooms.

Geographical Analysis in EDA:

1. State Distribution:

- We'll analyze how the **property listings are distributed across different states**. This will help us understand the geographic spread and any potential regional focus.

2. Price Distribution by State:

- We will examine the **average price by state** to understand which regions have the highest and lowest property prices.

3. Price Distribution by City:

- Similarly, we will look at how **prices vary by city** within the states, which may provide more localized insights.

4. Geospatial Distribution of Properties:

- Plotting the **latitude and longitude** of properties on a map can help identify any clusters of listings and trends related to specific geographic areas.

Let's start with the steps:

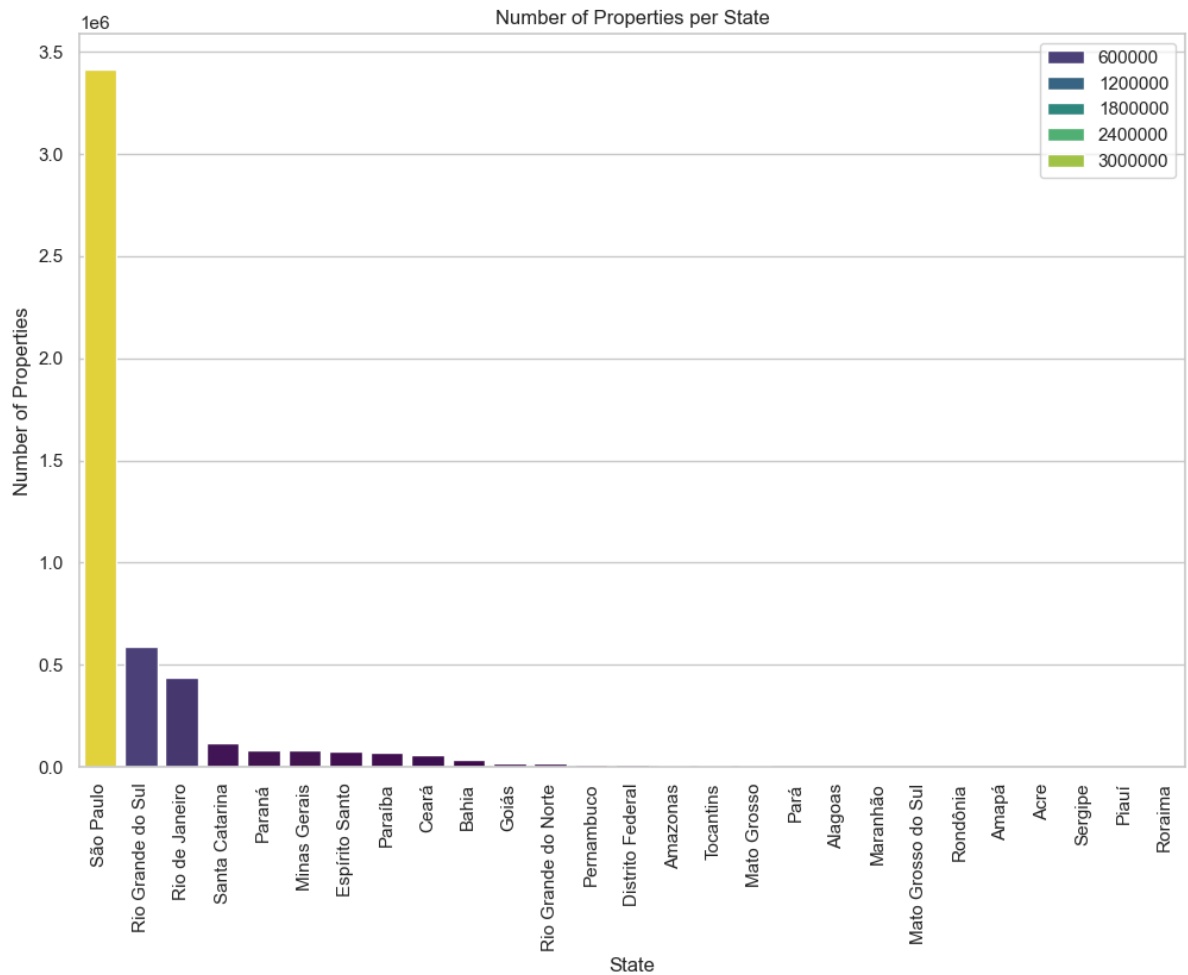
7. State Distribution

We will count the number of listings per state to understand where most properties are located.

```
In [29]: # Count the number of listings per state
state_counts = df['state_name'].value_counts()
state_counts
```

```
Out[29]: state_name
São Paulo          3415386
Rio Grande do Sul  592292
Rio de Janeiro     436918
Santa Catarina     118206
Paraná             83164
Minas Gerais       82869
Espírito Santo     78806
Paraíba            72915
Ceará              60521
Bahia              34900
Goiás              20463
Rio Grande do Norte 19934
Pernambuco         15164
Distrito Federal   11035
Amazonas           7328
Tocantins          6840
Mato Grosso        6761
Pará               4109
Alagoas            2286
Maranhão           1321
Mato Grosso do Sul  965
Rondônia           468
Amapá              383
Acre               124
Sergipe            90
Piauí              27
Roraima            4
Name: count, dtype: int64
```

```
In [30]: # Plot the state distribution
plt.figure(figsize=(12, 8))
sns.barplot(x=state_counts.index, y=state_counts.values, palette='viridis',
plt.title('Number of Properties per State')
plt.xlabel('State')
plt.ylabel('Number of Properties')
plt.xticks(rotation=90)
plt.show()
```



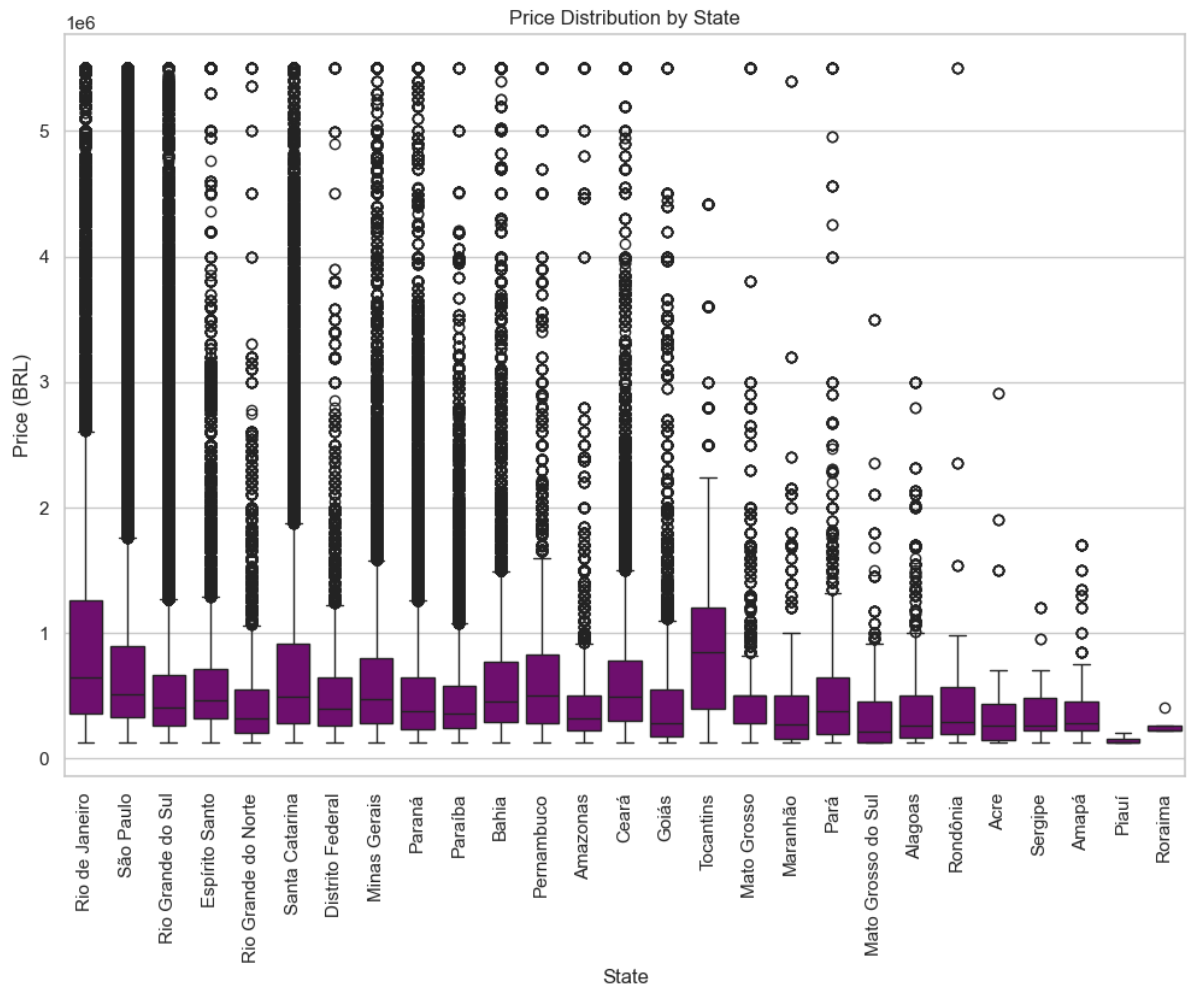
State Distribution

- The **state distribution bar chart** shows that **São Paulo** has the highest number of listings, which is expected given the size and population of the state.
- Other states with a high number of listings include **Rio de Janeiro**, **Rio Grande do Sul**, and **Santa Catarina**.

8. Price Distribution by State

Next, we'll examine the **price distribution by state** to see if some regions are more expensive than others.

```
In [31]: # Plot price distribution by state
plt.figure(figsize=(12, 8))
sns.boxplot(x='state_name', y='price', data=df, color='purple')
plt.title('Price Distribution by State')
plt.xlabel('State')
plt.ylabel('Price (BRL)')
plt.xticks(rotation=90)
plt.show()
```



Price Distribution by State

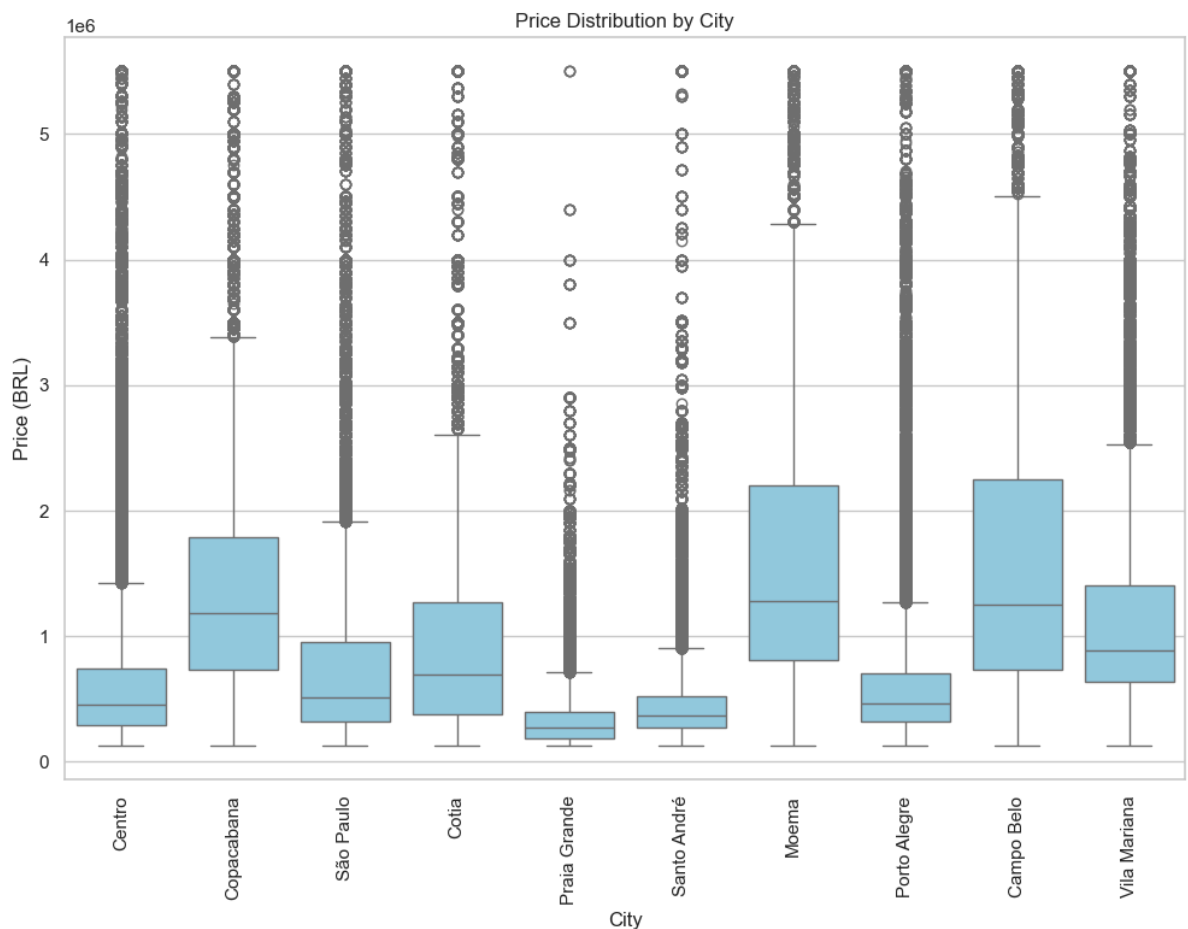
- The **price distribution by state** shows that states like **Rio de Janeiro** and **São Paulo** have a **wider range of property prices**, with several luxury listings. In contrast, other states like **Pernambuco** and **Acre** show more **concentrated price distributions** with lower median prices.

9. Price Distribution by City

Now, we will plot the price distribution by **city** within each state to understand any localized pricing differences limiting cities plotted by selecting a subset of cities with the highest number of listings.

```
In [32]: # Limit the number of cities to the top 10 most frequent cities
top_cities = df['place_name'].value_counts().nlargest(10).index
df_top_cities = df[df['place_name'].isin(top_cities)]

# Plot price distribution by city (for top 10 cities)
plt.figure(figsize=(12, 8))
sns.boxplot(x='place_name', y='price', data=df_top_cities, color="skyblue")
plt.title('Price Distribution by City')
plt.xlabel('City')
plt.ylabel('Price (BRL)')
plt.xticks(rotation=90) # Rotate the city names for better readability
plt.show()
```



Price Distribution by City

- The **price distribution by city** analysis shows that cities like **Copacabana**, **São Paulo**, and **Centro** have high price variability, with luxury properties driving up the upper limits of the price range.
- Cities like **Campo Belo** and **Moema** also show variability but with a lower price distribution than the larger cities.

Count the number of unique cities and states

```
In [33]: # Count the unique cities and states in the dataset
unique_cities = df['place_name'].nunique()
unique_states = df['state_name'].nunique()

print(f"Number of unique cities: {unique_cities}")
print(f"Number of unique states: {unique_states}")
```

Number of unique cities: 5034

Number of unique states: 27

Create a visualization showing cities within each state

We can create a grouped bar chart to display the number of cities in each state.

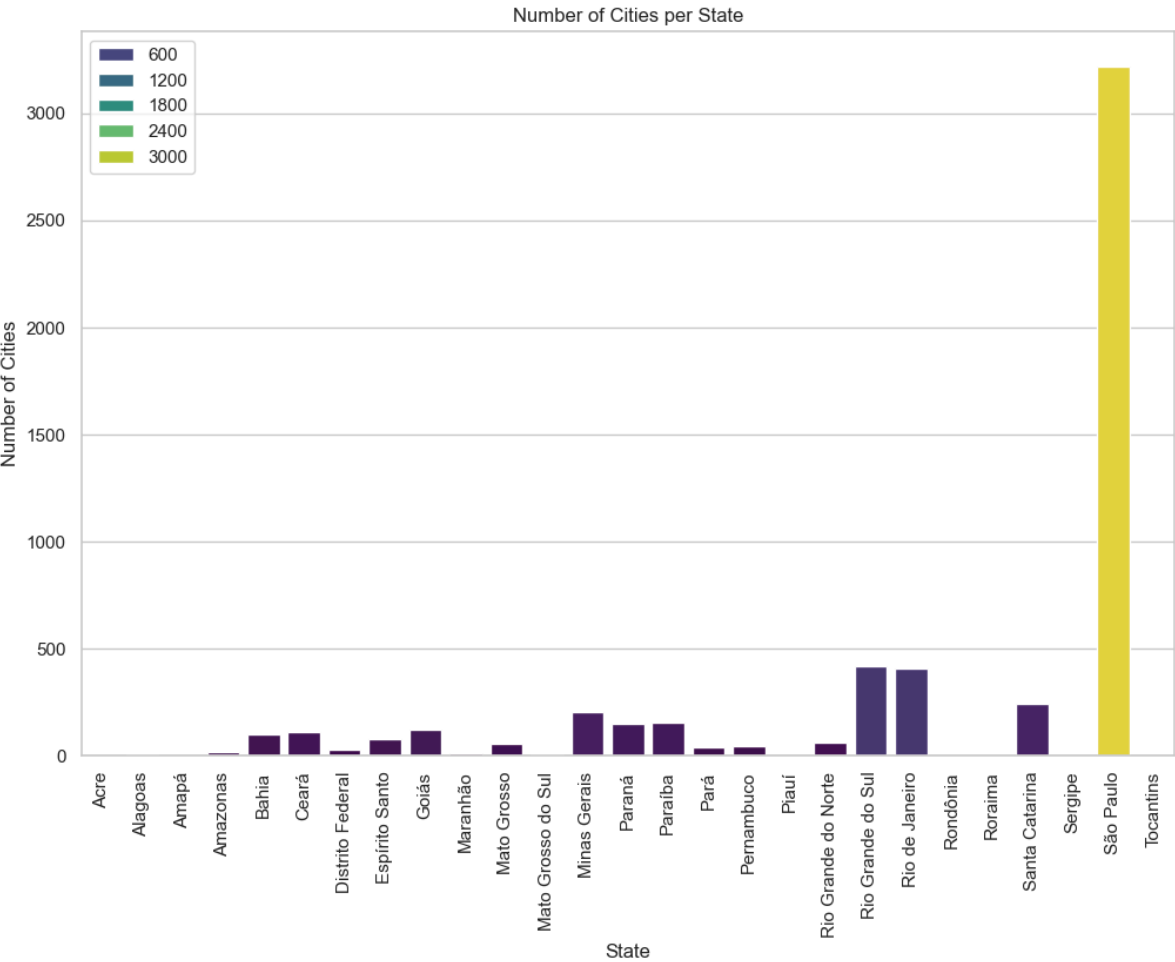
```
In [34]: # Count the number of cities in each state
city_counts_per_state = df.groupby('state_name')['place_name'].nunique()

city_counts_per_state
```



```
Out[34]: state_name
Acre 4
Alagoas 8
Amapá 2
Amazonas 17
Bahia 98
Ceará 111
Distrito Federal 28
Espírito Santo 80
Goiás 120
Maranhão 11
Mato Grosso 59
Mato Grosso do Sul 6
Minas Gerais 202
Paraná 148
Paraíba 153
Pará 41
Pernambuco 44
Piauí 1
Rio Grande do Norte 64
Rio Grande do Sul 420
Rio de Janeiro 410
Rondônia 2
Roraima 1
Santa Catarina 241
Sergipe 2
São Paulo 3220
Tocantins 4
Name: place_name, dtype: int64
```

```
In [35]: # Plot the number of cities per state
plt.figure(figsize=(12, 8))
sns.barplot(x=city_counts_per_state.index, y=city_counts_per_state.values, p
plt.title('Number of Cities per State')
plt.xlabel('State')
plt.ylabel('Number of Cities')
plt.xticks(rotation=90)
plt.show()
```



Key Insights and Recommendations

Step 1: High-Demand Areas and Property Types

We will start by analyzing the demand and supply of different property types and their pricing trends. This involves determining which property types (e.g., apartments vs. houses) are more frequently listed and whether some areas (cities or states) show higher demand than others. We will also examine whether there is a correlation between property supply (number of listings) and pricing.

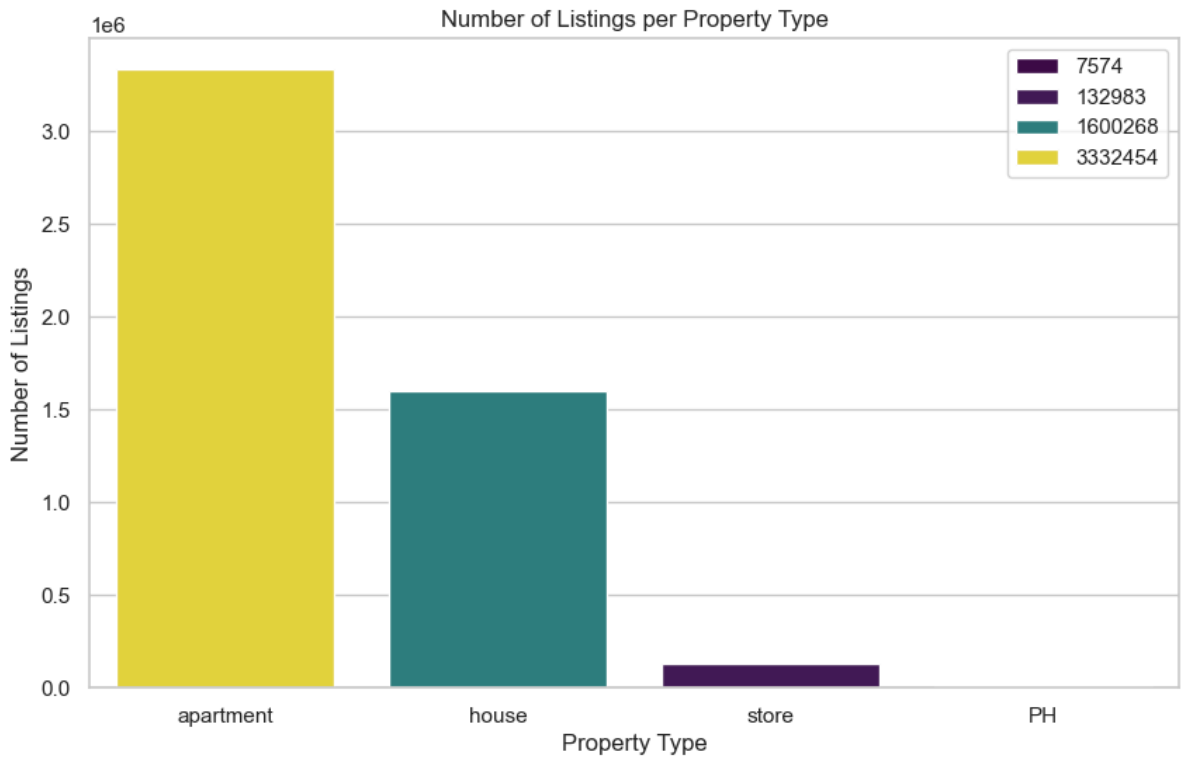
- **Most Popular Property Types:** The bar chart will help us understand which property types are most commonly listed in the dataset. For example, we may see that apartments dominate the listings in cities like São Paulo or Rio de Janeiro.
- **Demand vs. Supply in States:** By analyzing the number of listings by state, we can identify areas with higher demand (i.e., where more listings are available). If a region has a higher supply, we can check whether this is correlated with lower prices due to competition or if some property types are in high demand.

We can visualize the correlation between the number of listings per property type and the average price in those locations to understand if demand influences pricing.

Code for Analyzing Property Types and Locations:

We can start by counting the number of listings for each property type and location (state or city).

```
In [36]: # Count the number of listings by property type
property_type_counts = df['property_type'].value_counts()
# Plot the distribution of property types
plt.figure(figsize=(10, 6))
sns.barplot(x=property_type_counts.index, y=property_type_counts.values, palette='magma')
plt.title('Number of Listings per Property Type')
plt.xlabel('Property Type')
plt.ylabel('Number of Listings')
plt.show()
```

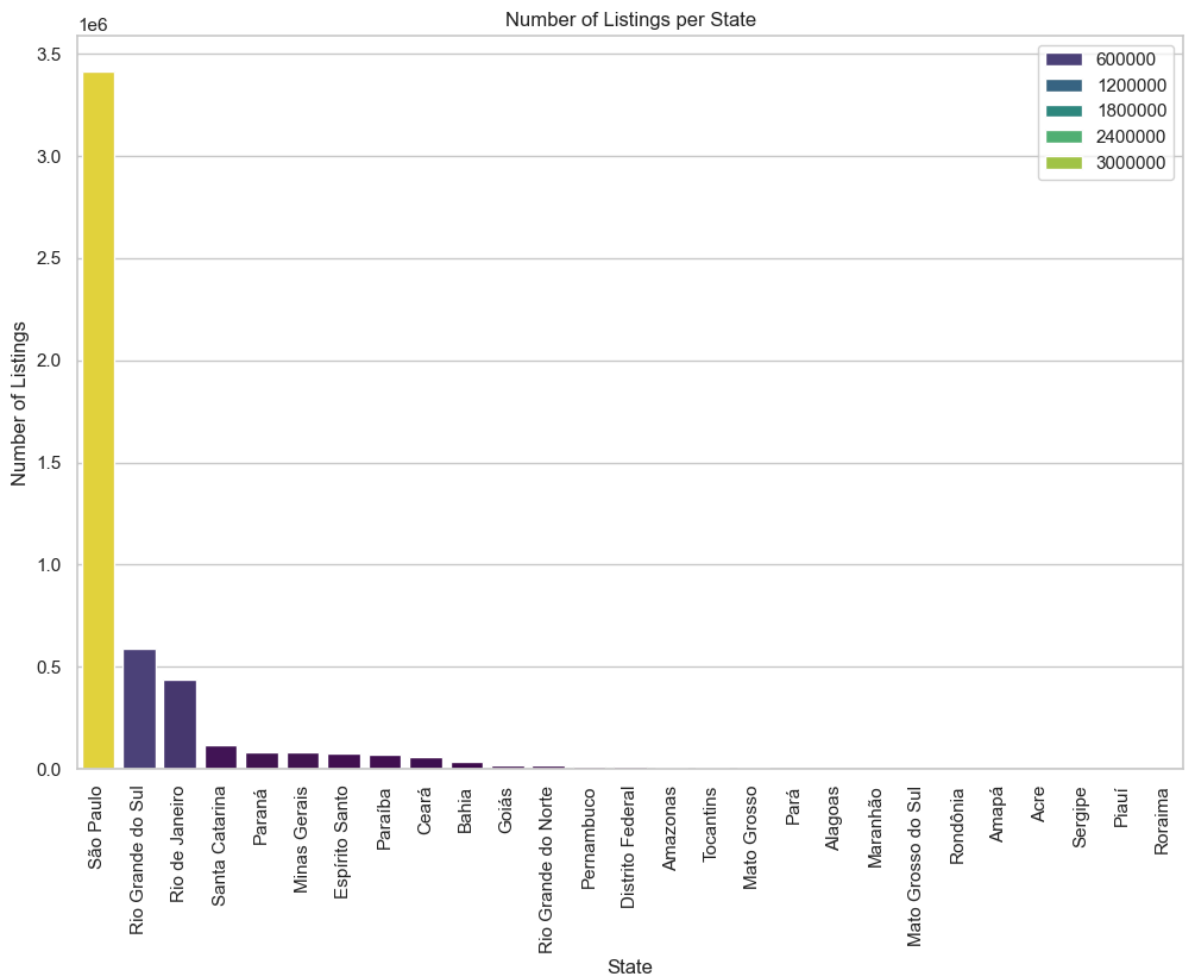


```
In [37]: # Count the number of listings by state
state_counts = df['state_name'].value_counts()
state_counts
```

```
Out[37]: state_name
São Paulo          3415386
Rio Grande do Sul  592292
Rio de Janeiro    436918
Santa Catarina    118206
Paraná            83164
Minas Gerais      82869
Espírito Santo    78806
Paraíba          72915
Ceará             60521
Bahia             34900
Goiás            20463
Rio Grande do Norte 19934
Pernambuco        15164
Distrito Federal  11035
Amazonas          7328
Tocantins         6840
Mato Grosso       6761
Pará              4109
Alagoas           2286
Maranhão          1321
Mato Grosso do Sul 965
Rondônia          468
Amapá             383
Acre              124
Sergipe           90
Piauí             27
Roraima           4
Name: count, dtype: int64
```

```
In [38]: # Plot the distribution of listings by state
plt.figure(figsize=(12, 8))
sns.barplot(x=state_counts.index, y=state_counts.values, palette="viridis",
plt.title('Number of Listings per State')
plt.xlabel('State')
```

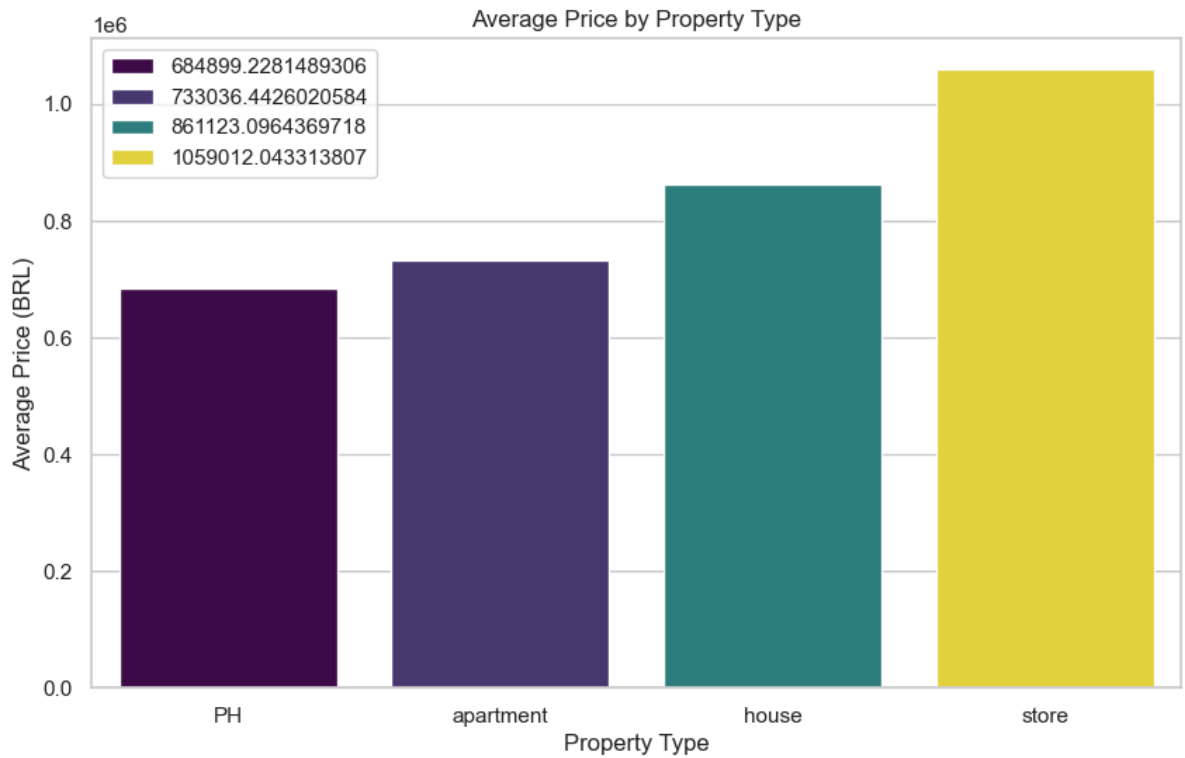
```
plt.ylabel('Number of Listings')
plt.xticks(rotation=90)
plt.show()
```



- **Price Trends by Property Type:** The bar plot will allow us to see which property types are more expensive on average. For instance, we might see that houses have a higher average price compared to apartments.
- **Price Trends by State:** The average price by state will show where the most expensive properties are located. This can help us identify regions that are in high demand, possibly due to location, infrastructure, or economic factors. ##### **Code for Pricing Trends by Property Type and State:**

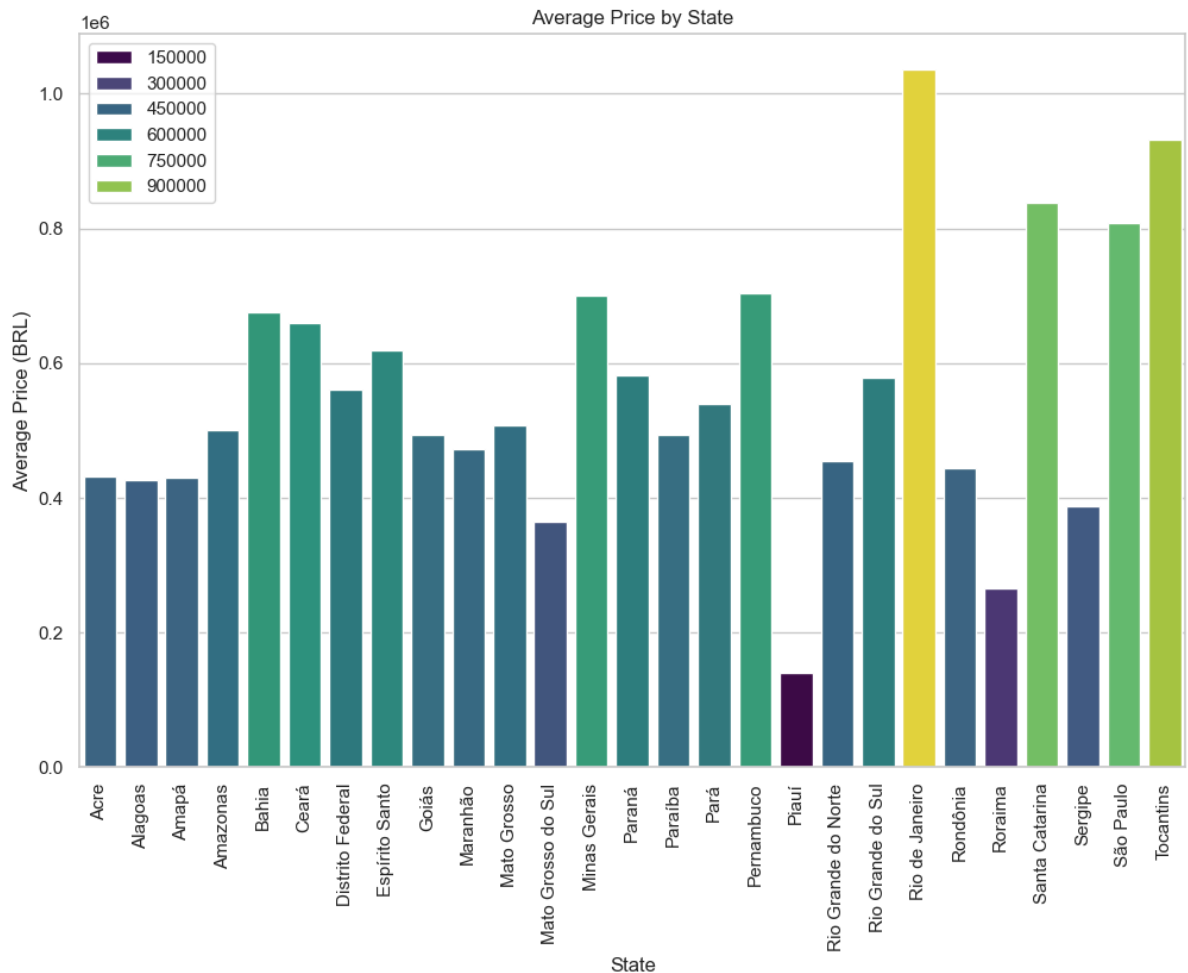
```
In [39]: # Calculate the average price per property type
avg_price_by_property_type = df.groupby('property_type')['price'].mean()

# Plot the average price by property type
plt.figure(figsize=(10, 6))
sns.barplot(x=avg_price_by_property_type.index, y=avg_price_by_property_type)
plt.title('Average Price by Property Type')
plt.xlabel('Property Type')
plt.ylabel('Average Price (BRL)')
plt.show()
```



```
In [40]: # Calculate the average price per state
avg_price_by_state = df.groupby('state_name')['price'].mean()

# Plot the average price by state
plt.figure(figsize=(12, 8))
sns.barplot(x=avg_price_by_state.index, y=avg_price_by_state.values, palette=
plt.title('Average Price by State')
plt.xlabel('State')
plt.ylabel('Average Price (BRL)')
plt.xticks(rotation=90)
plt.show()
```



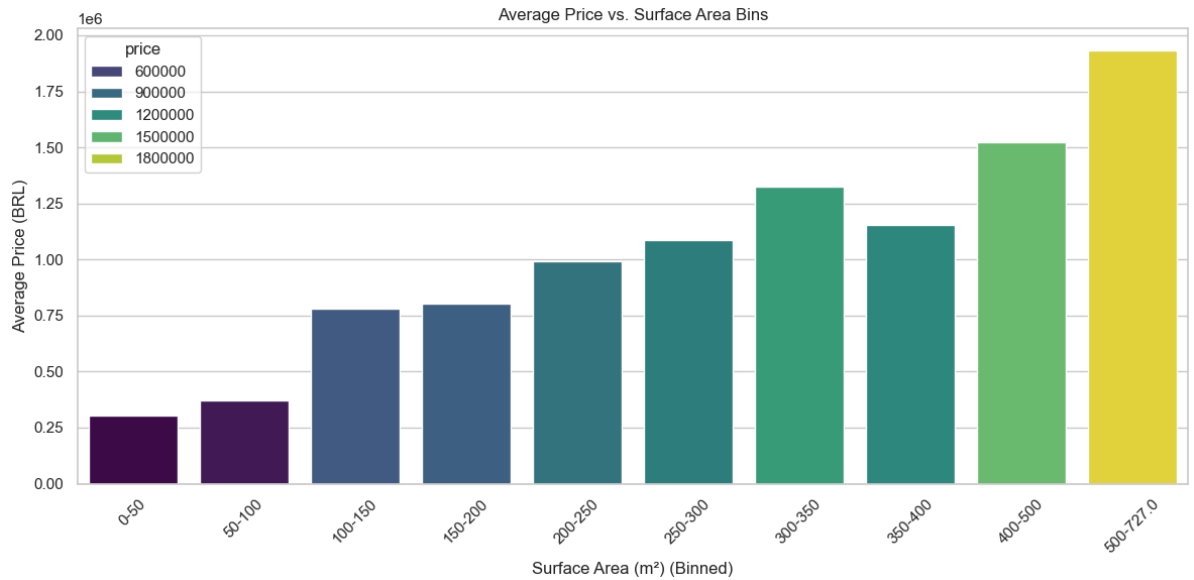
Step 2: Impact of Property Features on Price

Now, let's look at how different property features (e.g., surface area, number of rooms) affect the price. We will plot relationships between price and surface area and between price and the number of rooms to see if these features have a significant impact on the price.

- **Price vs. Surface Area:** The scatter plot between surface area and price will allow us to understand whether larger properties command higher prices. Larger properties tend to have higher prices, but we can check if there is any anomaly or clustering in the data.
- **Price vs. Number of Rooms:** By examining the relationship between the number of rooms and price, we can assess whether properties with more rooms typically have higher prices. This will help us identify if the number of rooms has a significant impact on pricing or if other features, such as surface area, play a larger role.

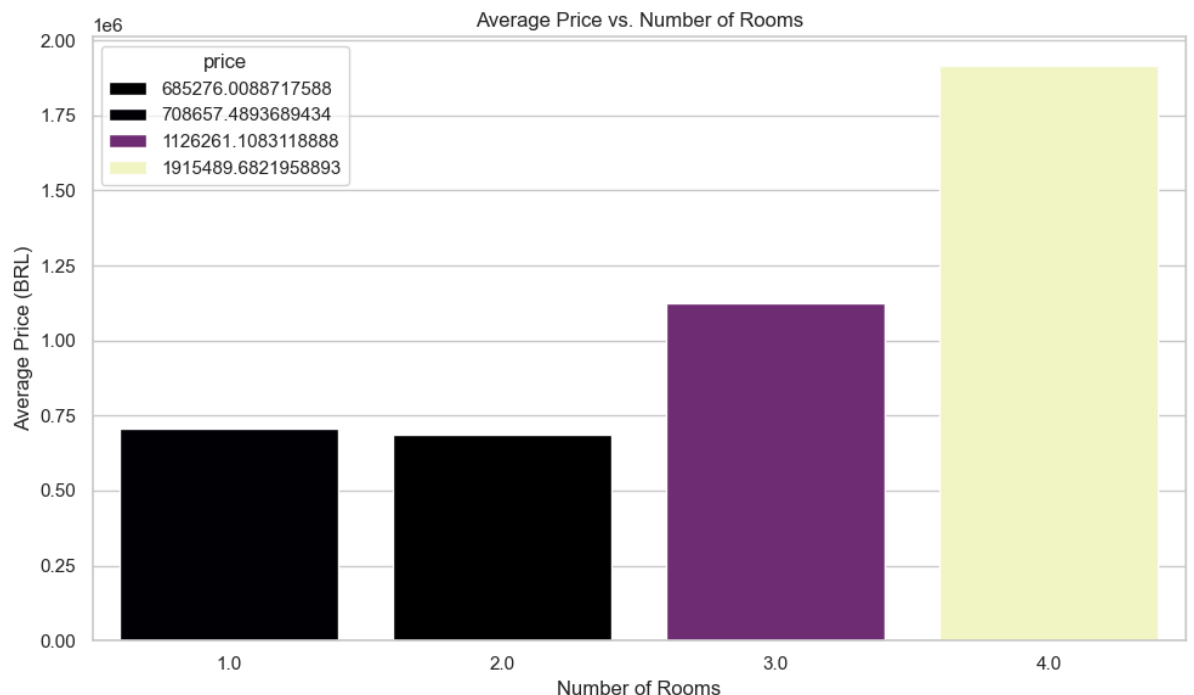
Code for Price vs. Surface Area and Price vs. Number of Rooms:

```
In [41]: plt.figure(figsize=(12, 6))
sns.barplot(x='surface_bin', y='price', data=avg_price_surface, palette='vi
plt.title('Average Price vs. Surface Area Bins')
plt.xlabel('Surface Area (m²) (Binned)')
plt.ylabel('Average Price (BRL)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
In [42]: # Group by the number of rooms and calculate the average price
avg_price_rooms = df.groupby('rooms')['price'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(x='rooms', y='price', data=avg_price_rooms, palette='magma', hue='rooms')
plt.title('Average Price vs. Number of Rooms')
plt.xlabel('Number of Rooms')
plt.ylabel('Average Price (BRL)')
plt.tight_layout()
plt.show()
```



Step 3: Emerging Trends and Investment Opportunities

Now, let's try to identify any emerging trends by examining regions with high growth in property prices, which can offer great investment opportunities. This involves analyzing the **price trends over time** and comparing them between states or cities.

- **Price Trends Over Time:** This line plot will show the overall price trend across the years in the dataset. By analyzing the trend, we can determine whether the market

has been appreciating, stagnating, or declining over time. Investors can look for markets that show consistent growth in prices.

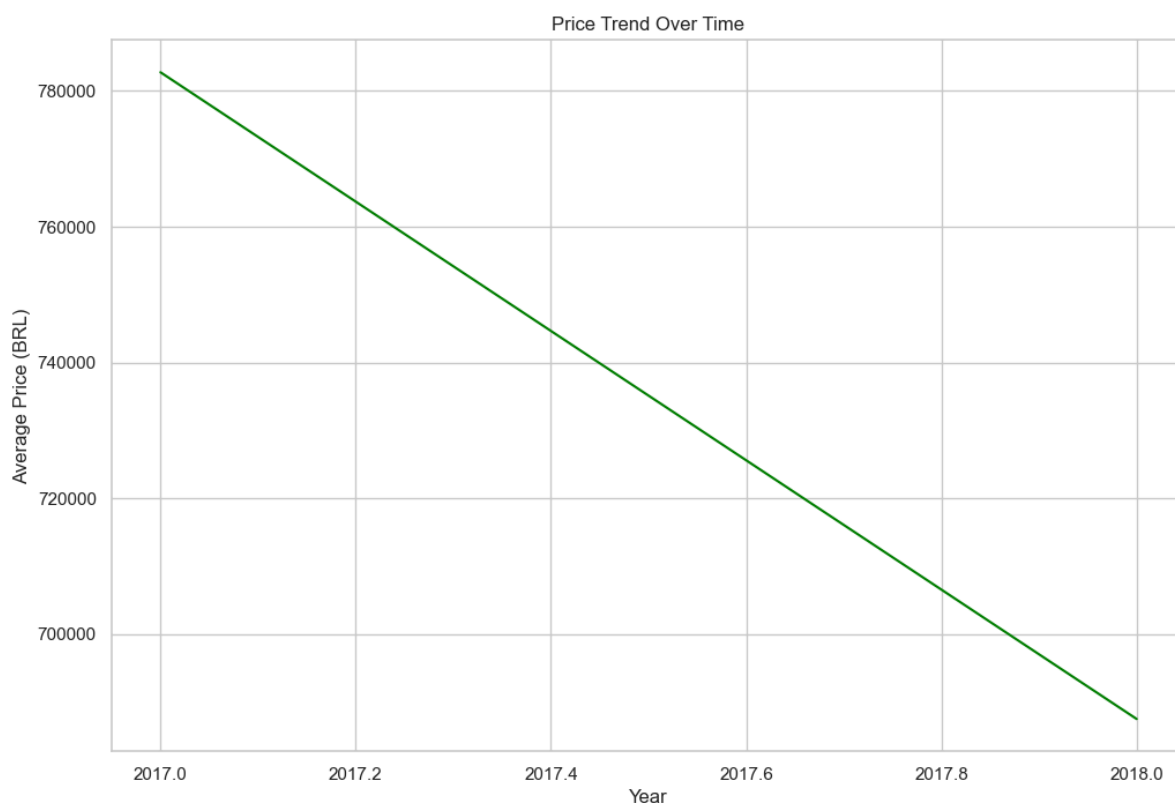
Code for Price Trend Over Time:

```
In [43]: # Convert 'created_on' to datetime format
df['created_on'] = pd.to_datetime(df['created_on'])

# Extract year from 'created_on'
df['year'] = df['created_on'].dt.year

# Calculate average price per year
avg_price_by_year = df.groupby('year')['price'].mean()

# Plot price trend over time
plt.figure(figsize=(12, 8))
sns.lineplot(x=avg_price_by_year.index, y=avg_price_by_year.values, color='g')
plt.title('Price Trend Over Time')
plt.xlabel('Year')
plt.ylabel('Average Price (BRL)')
plt.show()
```



Key Observations:

1. Median Price by State:

- São Paulo has the highest median property price, with a significant gap from other states. This indicates that São Paulo is a major economic hub with higher-value properties.
- States like Rio de Janeiro, Santa Catarina, and Rio Grande do Sul also have relatively high median property prices compared to the rest of Brazil, likely due to their economic importance or tourism-driven demand.

2. Surface Area Distribution:

- The distribution of surface area is heavily right-skewed, with most properties having smaller surface areas (around 100 m²).
- Larger properties (above 500 m²) are rarer, which confirms that most listings cater to the mid-range market in terms of size, with few ultra-large properties.

3. Rooms Distribution:

- The majority of properties have between 1 and 2 rooms, which suggests that small apartments or homes are more prevalent in the dataset.
- Outliers with larger room counts represent properties that likely cater to wealthier buyers or luxury markets.

4. Correlation Between Price, Surface Area, and Rooms:

- There is a moderate positive correlation between property price and both surface area and number of rooms. Larger properties with more rooms are priced higher, but the correlation isn't perfect, indicating other factors may play a role (such as location).
- This relationship suggests that both surface area and room count are significant in pricing, but a property's location, age, and condition also significantly impact the price.

5. Price vs. Surface Area and Price vs. Number of Rooms:

- The plots indicate that both larger surface area and more rooms generally correlate with higher prices. However, as the size increases, the price increase becomes less consistent, particularly in the ultra-large property range, suggesting a diminishing return for properties above certain sizes.

6. State and City Distribution:

- São Paulo has the highest number of property listings (over 3 million), reflecting its dominance as a real estate market hub.
- Other states like Rio de Janeiro, Minas Gerais, and Santa Catarina have fewer listings but still exhibit significant activity. On the other hand, states like Piauí, Acre, and Rondônia have far fewer listings, which could reflect less demand or less economic activity.
- Cities such as Copacabana (in Rio de Janeiro) and São Paulo show high price variability, indicating the presence of both luxury properties and more affordable ones within the same city.

Recommendations Based on Observations:

1. For Investors:

- Focus on metropolitan areas like **São Paulo** and **Rio de Janeiro** where demand and listings are high. These areas will offer a good balance of both high-value properties and opportunities for returns.
- **Minas Gerais** and **Santa Catarina** could be considered for mid-range investments, as these regions are growing in demand but still offer more affordable entry points than São Paulo or Rio de Janeiro.

2. For Developers:

- In high-demand areas like **São Paulo**, **Rio de Janeiro**, and **Santa Catarina**, developers can target luxury markets with properties exceeding 200 m² or more, which are likely to appeal to wealthier buyers.
- Developers looking to target more affordable housing could focus on areas in **Minas Gerais**, **Espírito Santo**, or **Ceará**, where there is strong demand but relatively lower price points compared to major cities.

3. For Property Buyers:

- Buyers looking for high-value properties should focus on cities with higher average prices and larger listings, such as **São Paulo** or **Rio de Janeiro**, where both property size and price can fluctuate considerably, allowing more flexibility in finding the right property.
- Buyers in more affordable markets, such as those in **Pernambuco** or **Paraíba**, should focus on areas with lower property prices for long-term value appreciation, especially if they are looking for starter homes or mid-range investments.

4. For Real Estate Agents and Marketers:

- Focus marketing efforts on high-demand metropolitan areas. With **São Paulo** and **Rio de Janeiro** having such a high number of listings and price variability, targeted campaigns emphasizing value and luxury will be effective in these regions.
- In cities with lower listings but emerging growth, such as **Ceará** or **Bahia**, advertising more affordable housing options or investment properties could attract potential buyers.

Conclusion

This analysis of the Brazilian real estate market provides a comprehensive view of the property listings across various states, property types, and time periods, helping to uncover significant trends and patterns. Key insights show that:

- **São Paulo** dominates the market, not only in the number of listings but also in higher median property prices, reinforcing its position as the country's economic hub.
- **Price distribution** reveals that apartments and houses are the most common listings, with penthouses and stores catering to niche markets. The variance in property prices by state suggests strong regional differences, with urban centers like São Paulo and Rio de Janeiro commanding the highest prices.
- **Surface area** and the **number of rooms** are positively correlated with higher prices, though the influence of these factors can vary. Larger properties tend to cost more, but this doesn't always equate to luxury pricing, especially in areas outside the major metropolitan cities.
- **Price trends** over time showed a slight downward movement in property prices from 2017 to 2018, suggesting some market adjustments. However, without clear causal factors, the future of the market will depend on broader economic conditions.

These findings offer valuable insights for investors, developers, and property buyers to make informed decisions based on regional demand, property type, and pricing trends. Investors may want to focus on high-demand urban areas like São Paulo and Rio de Janeiro, while developers could benefit from tapping into growing demand for affordable housing in less saturated regions.

In summary, the Brazilian real estate market is dynamic and offers various opportunities depending on the location, property type, and market conditions. While there are areas of price decline, overall, the market remains significant for investment, development, and residential purposes.